OPERATION KOMOREBI

Projector-Resolved Seed Calculus I

Machine-native paper

Records, semantics, executable actions,
and evidence contracts

Komorebi P.

Final editorial edition 5 October 2026
Companion to corrected machine interface v1.0.1

---

108 source-linked records / 69 attached source proofs
20 typed signatures / 8 runtime actions
Full record catalog and reproducible exact examples

Edition statement

This volume presents the machine representation of Paper I as a mathematical object that can be inspected, queried, and executed. It accompanies the traditional paper and the Lean guide. The canonical machine object remains the JSON entrypoint and its versioned payload. The PDF is the readable specification and catalog of that object.

The editorial edition is final for this release. This means its terminology, package paths, runtime description, examples, and record catalog have been reconciled with the v1.0.1 interface. It does not mean every record has a formal proof certificate. The release contains 108 records, including definitions and boundaries, and 69 source proof bodies. Twenty records have explicit typed signatures. Eight records have mappings to the attached historical Lean tranche, with differing degrees of source agreement.

The traditional final edition integrates the expanded 108-record core with worked models and visual material from the corrected traditional lineage. The machine source is the expanded core with two explicit nonzero-packet hypotheses added after the scoped audit. Its new SHA-256 digest identifies this corrected core. The original v1.0.0 payload is preserved separately as provenance. Added traditional examples are not silently inserted into the KIR. Stable identifiers join the core across the companions. A shared identifier alone is not a proof of statement equivalence.

The current documented release gate, exact finite fixture suite, and narrow zero-packet regression were replayed successfully for this edition. No fresh Lean compilation is claimed by this companion build. The separate Lean guide records its own formal verification scope.

How to use this volume. Read Sections 1–3 for mathematical and evidence semantics. Sections 4–6 give the executable contracts and an exact worked example. Section 7 describes release reproduction. Appendix A reproduces every canonical source statement and every attached proof body, with source locators and machine metadata. Later appendices give signatures, action coverage, operation vocabulary, and the historical Lean map.

Contents

Edition statement **1**

1 The mathematical object behind the interface **4**

1.1 Operator systems, generated hulls, and seed rank 4

1.2 Exact packet coordinates 4

1.3 Transfer blocks and composition 4

1.4 Intertwining and reconstruction 5

1.5 The boundary of finite execution 5

2 Canonical machine representation **5**

2.1 Entrypoint and package identity 5

2.2 Record identity and source provenance 6

2.3 Statement AST and typed signatures 6

2.4 Dependencies and closure 6

2.5 Operation profiles and actions 6

3 Evidence semantics and certification scope **7**

3.1 Independent evidence coordinates 7

3.2 Historical Lean linkage 7

3.3 Current reproduction 7

3.4 What PASS means 7

4 Executable action contracts **8**

4.1 Input and output conventions 8

4.2 The six exact finite actions 8

4.3 The two semantic gate actions 8

4.4 Planner-only and fixture-only actions 9

5 An exact worked execution **9**

5.1 Oblique packets and reconstruction 9

5.2 Resolving and rebuilding an operator 10

5.3 Composing resolved operators 10

5.4 An intertwining computation 10

5.5 Inspecting the dependency set 11

6 Transport profiles in use **11**

6.1 Finite packet multiplication 11

6.2 A countable result is blocked 12

6.3 Changing a profile without changing evidence 12

7 Reproduction and release maintenance **13**
7.1 Documented payload checks . . . 13
7.2 Companion build . . . 13
7.3 A source change is a new evidence task . . . 13
7.4 Archived interfaces and current prose . . . 13

A Complete source-linked record catalog **14**
A.1 Source section 1: Coefficient conventions, logical status, and scope . . . 15
A.2 Source section 2: Typed operator grammars, realizations, and seed systems . . . 16
A.3 Source section 3: Free presentations by seeds and relations . . . 21
A.4 Source section 4: Seed rank and functorial calculus . . . 25
A.5 Source section 5: Generator number modulo the Jacobson radical . . . 32
A.6 Source section 6: Single-operator modules over arbitrary fields . . . 37
A.7 Source section 7: Exact finite and countable packet coordinates . . . 40
A.8 Source section 8: Transfer-block algebra and support patterns . . . 50
A.9 Source section 9: Transfer graphs and directed packet generation . . . 59
A.10 Source section 10: Coordinate transport, morphisms, invariant exact sequences, and completion . . . 65
A.11 Source section 11: Realization functors and semantic theorem transport . . . 73
A.12 Source section 12: Canonical export interface and claim boundary . . . 76

B Typed core signatures **78**

C Complete action registry **83**

D Operation and hypothesis vocabulary **89**

E Historical Lean record map **91**

F Current verification receipt **93**

1 The mathematical object behind the interface

1.1 Operator systems, generated hulls, and seed rank

A seed problem consists of an acting algebra, a module, and a family of seed vectors. The algebra must be fixed before the generation question is asked. For a unital algebra A acting on a module M and a finite seed family $S = \{s_1, \dots, s_r\}$, the algebraic generated hull is

$$\mathrm{Hull}_A(S) = \sum_{j=1}^{r} As_j.$$

A one-seed system is cyclic. Seed rank measures the minimum cardinality of a generating family, under the regime declared in the source. Enlarging the acting algebra changes the question. Replacing algebraic span by a closure changes it again. The machine record therefore keeps the acting regime and hypotheses beside the statement.

Paper I begins with typed operator grammars and their realizations, then passes through free presentations, generator number, packet coordinates, block algebras, transfer graphs, and realization. The resulting record graph covers the whole expanded foundation. It is not limited to the finite matrix examples that the runtime executes.

For finite families of nonnegative integers, the ambient convention is $\max \varnothing = 0$. The height of an empty finite poset is zero. A nonempty poset with no strict comparable pair also has height zero, so height counts strict steps. These conventions clarify the empty cases without adding stable records.

The source separates algebraic, linear, and topological claims. A finite-dimensional matrix computation can exercise a linear identity. It cannot establish the countable Banach-space result or a generator formula over every division ring. Those claims retain their own source proofs and dependencies.

1.2 Exact packet coordinates

For a finite index set I, a packet system on X consists of linear maps P_α satisfying

$$P_\alpha^2 = P_\alpha, \qquad P_\beta P_\alpha = 0 \quad (\beta \neq \alpha), \qquad \sum_{\alpha \in I} P_\alpha = I_X.$$

The packet spaces are the ranges $X_\alpha = P_\alpha X$. Analysis and synthesis are

$$\mathcal{A}_P x = (P_\alpha x)_{\alpha \in I}, \qquad \mathcal{S}_P((x_\alpha)_\alpha) = \sum_{\alpha} x_\alpha.$$

Their inverse laws are exact. No orthogonality assumption is needed. This distinction is visible in the oblique two-dimensional example later in the paper.

Machine execution represents each packet as an ambient vector. It does not construct a basis of every range or replace the ambient representation by smaller coordinate arrays. This choice makes exact identities easy to compare with the source formulas while retaining the packet index structure.

1.3 Transfer blocks and composition

For an endomorphism T, the transfer block from packet α to packet β is

$$T_{\beta\alpha} = P_\beta T P_\alpha.$$

Its source is X_α and its target is X_β. In the finite runtime it is stored as a full ambient matrix. Completeness gives

$$T = \sum_{\alpha,\beta} T_{\beta\alpha}, \qquad (ST)_{\gamma\alpha} = \sum_{\beta} S_{\gamma\beta} T_{\beta\alpha}.$$

The order matters: the rightmost factor acts first. The runtime uses the same convention. Its nested block array is indexed first by target and then by source.

The graph records source dependencies for these identities. It does not infer every implicit mathematical dependency. The packet-system hypotheses remain required even when a compact source statement inherits them from the surrounding section.

1.4 Intertwining and reconstruction

For packet systems P on X and Q on Y, an intertwiner $U : X \to Y$ satisfies $UT = SU$. Resolving both sides yields

$$\sum_{\beta} U_{\gamma\beta} T_{\beta\alpha} = \sum_{\delta} S_{\gamma\delta} U_{\delta\alpha}, \qquad U_{\gamma\alpha} = Q_\gamma U P_\alpha.$$

All local equations together are equivalent to the global equation under the packet hypotheses. The runtime returns the global Boolean, every local residual, and the conjunction of the local Booleans. It provides the data needed to inspect agreement or failure in a concrete exact case.

1.5 The boundary of finite execution

Countable packet sums require a topology and a convergence condition. Quotient descent requires invariance. Coordinate transport requires a suitable invertible map. Packet-pure seed counts differ from unrestricted seed counts because one mixed-support seed can contribute to several source components. These are mathematical hypotheses, not formatting conventions.

The source records seven explicit boundaries. Their presence is part of the interface. Runtime action names do not imply that every associated theorem has an implementation, and metadata listing a boundary does not itself execute that boundary check.

2 Canonical machine representation

2.1 Entrypoint and package identity

The payload is located at `source/machine_v1.0.1/`. Commands in this volume assume the working directory is that folder. The entrypoint is `PSC_I.machine.json`. It points to the record collection, statement AST, dependency graph, action registry, evidence index, and release files.

The frozen source digest is:

```
37bf8f03dbbf66ce41f8b07a8e7be8fa5fca44176bdd56dd25b483f72736ff77
```

The digest identifies the corrected expanded TeX source. The only mathematical statement changes from v1.0.0 are explicit nonzero-packet assumptions in the adjoint-closed support-pattern corollary and the unweighted source-component-count corollary. Both require nonzero packets for their necessity or counting arguments. An unnumbered paragraph also declares empty-family and poset-height conventions. All proof bodies are unchanged. The two degenerate examples are retained in the patch regression. A reformatted PDF or an integrated traditional edition need not have that digest. Reusing the frozen machine evidence requires retaining its source identity or recording an explicit crosswalk to a changed source.

2.2 Record identity and source provenance

Each record has a stable ID, kind, title, source section, statement text, optional proof text, dependencies, semantic annotations, evidence coordinates, and fingerprint. The source locator includes statement and proof spans and their hashes. These fields make it possible to ask which statement was encoded and which proof text accompanied it.

There are 28 definitions, 28 theorems, 22 corollaries, 16 propositions, three lemmas, three classical inputs, seven boundaries, and one remark. The 69 theorem/proposition/corollary/lemma records have attached proof bodies. Definitions and boundary statements are not counted as proved theorems merely because they occupy records.

A record fingerprint provides a content identity under the release's generating convention. It is not a proof certificate. The release gate checks the source digest, ID count, graph, AST coverage, selected API behavior, and exact fixtures. It does not establish all mathematical content through independent formal reconstruction.

2.3 Statement AST and typed signatures

The statement AST has one entry per stable ID. The statuses are 20 `TYPED_CORE`, 81 `STRUCTURED_SYMBOLIC`, and seven `TEXTUAL_SEMANTIC`. A typed-core record points to a signature declaring parameters, requirements where supplied, and a return description.

The AST segments statements into prose and mathematical surfaces. Some expressions are recognized as relations or direct-sum expressions. Others retain `latex_expr` leaves. Even a typed-core record can contain these leaves. Consequently, "typed core" refers to the attached interface signature, not to a complete formal elaboration or a proof term.

The signature for block intertwining describes a logical equivalence between the global and local equations. Its source proof remains attached separately. The runtime can evaluate a finite instance. These are three different representations with three different evidentiary roles.

2.4 Dependencies and closure

The recorded graph contains 108 nodes and 97 internal edges. The release gate confirms that the recorded internal dependencies resolve and that the graph is acyclic. Direct dependencies are stored on each record. The API can also return a transitive closure.

Closure queries return sorted identifiers. They do not produce a proof order, carry out the proof, or resolve external classical inputs. A consumer can use the graph to inspect a prerequisite set and then obtain source statements and evidence for each record.

2.5 Operation profiles and actions

Each record carries source-derived operation metadata, with a transitive profile alongside the direct profile. The ontology distinguishes structural operations from hypothesis classes. These annotations support planning and inspection. They should not be interpreted as a verified extraction of every semantic operation used by an arbitrary proof.

The action registry contains 37 action names associated with 75 records. Eight names are implemented in the runtime. Other entries are marked planner-only or fixture-only. The source has broader mathematical scope than the executable subset.

3 Evidence semantics and certification scope

3.1 Independent evidence coordinates

The record evidence object separates source, KIR, Python, and Lean. The source coordinate identifies the authoritative source. The KIR coordinate records the release's validation status. Python evidence reports exact finite fixtures for 26 records. Lean evidence points to eight historical source mappings.

No coordinate upgrades another automatically. Exact arithmetic removes floating-point ambiguity from a finite computation. It does not quantify over arbitrary fields, modules, dimensions, or topological spaces. A mapped Lean declaration proves its own formal statement under its dependencies. Its relation to the source statement must still be read from the mapping.

3.2 Historical Lean linkage

The attached verification tranche records Lean 4.34.0-rc2 and a pinned Mathlib manifest. The map is dated 2 September 2026. Its eight mappings include definitions, generalized theorems, one-directional correspondence, and a related sufficient direction. The support-pattern algebra mapping is explicitly partial. It must not be counted as a full formal certificate for every direction of the source criterion.

A separate historical report records 35 of 35 obligations for manuscript v1.2.0. The frozen status marks that lineage as noncanonical for the current expanded source. The historical count and the eight explicit current-core mappings answer different questions.

3.3 Current reproduction

For this edition, the documented release verifier and exact fixture suite were run from the corrected payload. Both passed. The verification log is attached in the companion's `build/` directory. All 65 files extracted from the original machine archive retain their original bytes.

An initial invocation used an interpreter without SymPy and failed when the blockification action imported that dependency. The successful replay used an existing environment containing SymPy. No runtime action implementation was changed to resolve that environment failure. The two audited statement corrections were applied separately and have their own provenance record.

No fresh Lean replay is attributed to this companion build. Historical reports remain historical. If the separate Lean guide performs a fresh replay, it should identify the exact Lean source and toolchain that were tested rather than rewriting this frozen status file.

3.4 What PASS means

A release-gate pass means the named package checks succeeded. A fixture pass means a concrete exact computation satisfied its assertion. A transport-gate pass means all operation flags that this runtime checked were accepted. None of these strings, taken alone, means that the entire 108-record foundation has been formally proved by the runtime.

The full source proof catalog is included so that a reader can inspect mathematical arguments beyond the runtime. Editorial reproduction of those arguments is not an adversarial proof audit or a new formal verification claim.

4 Executable action contracts

4.1 Input and output conventions

The API is a Python command that emits JSON on successful calls. Exact arithmetic actions use SymPy matrices. Integers and rational strings such as `"3/7"` preserve rational values. JSON floating-point numbers should be avoided when the intended claim is an exact rational identity. Returned scalar entries are strings.

Payload shapes are conventional rather than comprehensively validated at the API boundary. Supply nonempty compatible square projector matrices and compatible operators or vectors. Malformed inputs can raise Python exceptions. The command parser does not present a universal structured-error contract.

Most computational actions do not call packet validation automatically. Validate the packet family before interpreting block or analysis output through the theorems. With invalid projectors, a matrix expression may still be computable while its theorem interpretation fails.

4.2 The six exact finite actions

Action	Payload	Returned information
`VALIDATE_PACKET_SYSTEM`	`projectors`	Idempotence, annihilation, completeness, and their conjunction.
`ANALYZE_PACKETS`	`projectors`, `vector`	Ambient packet vectors and a reconstruction Boolean.
`BLOCKIFY_OPERATOR`	`projectors`, `operator`	Target/source array of ambient transfer matrices.
`RECONSTRUCT_OPERATOR`	`blocks`	Sum of every supplied block. Compatibility with a packet family is not independently checked.
`COMPOSE_TRANSFER_BLOCKS`	`projectors`, `S`, `T`	Block-product array and equality with directly resolved ST.
`LOWER_INTERTWINER`	`P`, `Q`, `T`, `S`, `U`	Global equation, local equations, residuals, and local conjunction.

4.3 The two semantic gate actions

`CHECK_REALIZATION_PROFILE` and `TRANSPORT_THEOREM` share the same runtime gate. Their payload contains a record ID and a profile. The gate reads that record's direct `action_required_operations` and checks the corresponding profile flags.

The accepted flags are true, `VERIFIED`, or `PRESERVED`. False and `NOT_PRESERVED` produce a negative list. Other or missing values are unknown. Any negative or unknown required operation blocks the gate.

This v1.0.1 implementation does not evaluate the profile's `verified_hypotheses`, does not use the transitive operation closure as its required set, and does not execute the returned boundary guards. It also does not construct a target theorem, a proof object, or an independently verified realization functor. The preservation flags are caller-supplied declarations.

Thus the action named `TRANSPORT_THEOREM` is an operation-profile gate. A consumer must separately justify target hypotheses, source-to-target interpretation, and any constructions used

by the proof. Unknown flags fail closed only within the operation list actually inspected by the function.

4.4 Planner-only and fixture-only actions

The full registry in Appendix C makes execution status explicit. A planner-only action can identify a mathematically meaningful next step without implementing it. A fixture-only association records that a finite check exercises related mathematics. Calling an unimplemented action through `execute` exits with an unsupported-action error.

5 An exact worked execution

5.1 Oblique packets and reconstruction

Use

$$P_0 = \begin{pmatrix} 1 & -1 \\ 0 & 0 \end{pmatrix}, \qquad P_1 = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}, \qquad x = \begin{pmatrix} 3 \\ 5 \end{pmatrix}.$$

These are complementary idempotents but are not self-adjoint under the usual Euclidean inner product. The example therefore exercises the algebraic packet hypotheses without adding orthogonality.

```
python runtime/machine_api.py execute VALIDATE_PACKET_SYSTEM tests/packet_input.json
python runtime/machine_api.py execute ANALYZE_PACKETS tests/analyze_input.json
```


```
{
  "complete": true,
  "idempotent": true,
  "pairwise_annihilating": true,
  "valid": true
}
```


The analysis output is exact:


```
{
  "packets": [
    [
      [
        "-2"
      ],
      [
        "0"
      ]
    ],
    [
      [
        "5"
      ],
      [
        "5"
      ]
    ]
  ],
  "reconstructs": true
}
```


The two vectors $(-2, 0)^t$ and $(5, 5)^t$ sum to x. Each belongs to its own projector range. The runtime's zero-based packet indexing corresponds to the displayed P_0, P_1.

5.2 Resolving and rebuilding an operator

Take

$$T = \begin{pmatrix} 2 & 3 \\ 5 & 7 \end{pmatrix}.$$

The command `BLOCKIFY_OPERATOR` returns four ambient 2×2 matrices. In target/source order, they are

$$T_{00} = \begin{pmatrix} -3 & 3 \\ 0 & 0 \end{pmatrix}, \quad T_{01} = \begin{pmatrix} 0 & -7 \\ 0 & 0 \end{pmatrix}, \quad T_{10} = \begin{pmatrix} 5 & -5 \\ 5 & -5 \end{pmatrix}, \quad T_{11} = \begin{pmatrix} 0 & 12 \\ 0 & 12 \end{pmatrix}.$$

Their sum is T. The exact reconstruction API returns:


```
{
  "operator": [
    [
      "2",
      "3"
    ],
    [
      "5",
      "7"
    ]
  ]
}
```


The input and output JSON files for this reconstruction are supplied in `examples/`. The reconstruction action performs the block sum. Its interpretation as recovery of the original operator uses the validated packet system and the way those blocks were formed.

5.3 Composing resolved operators

Now set

$$S = \begin{pmatrix} 11 & 13 \\ 17 & 19 \end{pmatrix}, \qquad ST = \begin{pmatrix} 87 & 124 \\ 129 & 184 \end{pmatrix}.$$

The runtime forms the blocks of S and T, then sums over the middle packet index. It compares the resulting array with the directly resolved product. The supplied worked output has `equals_direct: true`. The full block arrays are retained in `examples/composition.json`.

The comparison checks a concrete instance of the multiplication identity. The general identity follows from inserting the complete family between the factors, as in the source proof reproduced in Appendix A.

5.4 An intertwining computation

Let

$$U = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \qquad Q_i = UP_iU^{-1}, \qquad S' = UTU^{-1} = \begin{pmatrix} 7 & 3 \\ 5 & 2 \end{pmatrix}.$$

The target packets are $Q_0 = \left(\begin{smallmatrix} 1 & -2 \\ 0 & 0 \end{smallmatrix}\right)$ and $Q_1 = \left(\begin{smallmatrix} 0 & 2 \\ 0 & 1 \end{smallmatrix}\right)$. The global identity is $UT = S'U$.

The recorded execution of `LOWER_INTERTWINER` returns true for both the global identity and the conjunction of the four local equations. Each local residual is the zero matrix. One complete local result is:


```
{
  "holds": true,
  "lhs": [
    [
```

```
      "-3",
      "3"
    ],
    [
      "0",
      "0"
    ]
  ],
  "residual": [
    [
      "0",
      "0"
    ],
    [
      "0",
      "0"
    ]
  ],
  "rhs": [
    [
      "-3",
      "3"
    ],
    [
      "0",
      "0"
    ]
  ],
  "source": 0,
  "target": 0
}
```


This example deliberately changes packet coordinates as well as the operator. The complete input and four outputs are retained in `examples/intertwining_input.json` and `examples/intertwining.json`.

5.5 Inspecting the dependency set

The API exposes the dependencies used by the source-linked intertwining record:

```
python runtime/machine_api.py deps PSC-I-T-BLOCK-INTERTWINING --transitive
```


```
{
  "dependencies": [
    "PSC-I-D-ANALYSIS-SYNTHESIS",
    "PSC-I-D-PACKET-SYSTEM",
    "PSC-I-D-TRANSFER-BLOCK",
    "PSC-I-T-BLOCK-MULTIPLICATION",
    "PSC-I-T-EXACT-PACKET-COORDINATES",
    "PSC-I-T-OPERATOR-RECONSTRUCTION"
  ],
  "record": "PSC-I-T-BLOCK-INTERTWINING"
}
```


This is a dependency closure, not a newly generated proof. Reading each named record supplies its exact source statement, proof, and evidence coordinates.

6 Transport profiles in use

6.1 Finite packet multiplication

The packet-linear profile declares preservation of the direct operations required by packet multiplication. The command

```
python runtime/machine_api.py transport PSC-I-T-BLOCK-MULTIPLICATION profiles/packet_linear.json
```

returns:


```
{
  "boundary_guards": [],
  "not_preserved": [],
  "record": "PSC-I-T-BLOCK-MULTIPLICATION",
  "required_operations": [
    "composition",
    "finite_sum",
    "packet_idempotents",
    "transfer_blocks"
  ],
  "status": "PASS",
  "unknown": []
}
```


The pass is exactly a pass for the listed operation flags. In particular, it does not construct the target packet system or independently check a concrete functor's preservation laws.

6.2 A countable result is blocked

Using the same profile for the countable unconditional packet theorem returns:


```
{
  "boundary_guards": [],
  "not_preserved": [
    "completion",
    "topology"
  ],
  "record": "PSC-I-T-COUNTABLE-UNCONDITIONAL-PACKETS",
  "required_operations": [
    "completion",
    "packet_idempotents",
    "topology"
  ],
  "status": "BLOCKED",
  "unknown": []
}
```


The profile does not preserve the topological and completion operations required by this gate. A finite linear profile therefore cannot be used to pass this countable request.

The source theorem additionally requires a real or complex Banach space, unconditional reconstruction, and a uniform multiplier bound. These hypotheses remain visible in its statement. Supplying permissive operation flags would not prove those hypotheses. The richer source contract and the narrower executable gate must be kept distinct.

6.3 Changing a profile without changing evidence

A profile file is a declaration supplied to the runtime. Editing its flags changes the gate's output. It does not create mathematical preservation evidence. A downstream system that wants a transport certificate must retain the arguments or formal certificates justifying those flags and the target hypotheses.

The correct use is sequential: identify the exact record, inspect its source and dependency context, establish a target realization, justify the relevant preservation properties and hypotheses, then use the gate as an explicit consistency check. Its output can be attached to the resulting evidence record as one component.

7 Reproduction and release maintenance

7.1 Documented payload checks

From `source/machine_v1.0.1/`, use an interpreter containing SymPy:

```
python runtime/verify_release.py
python runtime/exact_fixture_suite.py
python runtime/nonzero_packet_regression.py
```

The first script checks entrypoint identity, record IDs, graph acyclicity and resolution, AST coverage, action-registry consistency, source hash, selected API queries and actions, structured transport response, and fixture replay. The second emits the detailed fixture results. The latter does not replace a theorem prover.

7.2 Companion build

From the machine companion directory, `build_companion.py` replays these checks, captures the worked examples, verifies every frozen file against the original archive, generates the TeX, and builds the PDF. The environment variable `PSC_PYTHON` can select a Python interpreter with SymPy. A TeX installation with the packages declared in the preamble is required for the PDF build.

The standalone source package includes a copy of the original machine release archive as provenance. The build accepts `PSC_MACHINE_ARCHIVE` when that archive is stored elsewhere. The document source itself remains readable independently of the builder.

7.3 A source change is a new evidence task

If a source statement changes, its source and statement identity change. The KIR, AST, dependency annotations, operation profile, formal map, and tests must be reviewed for impact. A historical pass cannot be assigned to a new statement solely because the stable ID stayed the same.

For this release, v1.0.1 records the two narrow statement corrections and synchronized machine metadata. The v1.0.0 payload remains byte-for-byte unchanged in provenance. Editorial additions are outside the machine payload. The additional traditional models and visual appendix belong to the human companion until their own machine records are explicitly defined and mapped.

7.4 Archived interfaces and current prose

The earlier v0.8 machine-native PDF is retained as provenance in the collected materials. This volume replaces its editorial account for the corrected v1.0.1 interface. In particular, it describes the present runtime's exact transport-gate scope and preserves the mathematical records apart from the two disclosed corrections, without relabelling planner metadata as executed proof machinery.

A Complete source-linked record catalog

The following catalog is generated from the 108 frozen KIR records. Statements and attached proof bodies are reproduced from v1.0.1. Relative to v1.0.0, exactly two statements gain an explicit nonzero-packet assumption. No proof body is changed. Original source cross-references are linked to the matching stable record in this catalog. This rendering changes navigation, not the source statement or proof.

The evidence shorthand in each record is read literally. “Mapped” indicates that a historical Lean mapping exists. It does not collapse the distinctions in Appendix E. A source proof is written mathematical evidence. A typed signature is an interface description. A fixture exercises a bounded case.

A.1 Source section 1: Coefficient conventions, logical status, and scope

1. Coefficient and module regimes

Stable ID. `PSC-I-D-COEFFICIENT-REGIME`

Kind and regime. definition; meta, algebraic, linear, topological

Statement.

All modules are left modules unless stated otherwise. The algebraic regime consists of a unital ring A and a unital left A-module M. When linear dimensions, tensor products, quivers, or operator blocks are used, A is a unital $\Bbbk$-algebra over a field $\Bbbk$ and M is a $\Bbbk$-vector space. The topological regime consists of a Hausdorff topological vector space over a complete valued field, together with a unital algebra of continuous linear operators. Every infinite sum, completion, quotient topology, and closure below is interpreted only in the regime explicitly declared in its statement. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 320–323.

2. Canonical theorem record

Stable ID. `PSC-I-D-THEOREM-RECORD`

Kind and regime. definition; meta

Statement.

A canonical theorem record is the tuple

$$(\mathrm{ID}, \mathrm{status}, \mathrm{hypotheses}, \mathrm{statement}, \mathrm{dependencies}, \mathrm{proof}, \mathrm{exports}, \mathrm{boundary}).$$

A downstream paper cites the stable record ID. A domain specialization may add domain hypotheses, but it does not repeat a generic proof from this paper. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 327–335.

3. Symbolic scope

Stable ID. `PSC-I-B-SYMBOLIC-SCOPE`

Kind and regime. boundary; meta, algebraic, linear, topological

Statement.

This edition proves the algebraic and topological statements printed below. It does not assert historical novelty, independent external certification, automatic discovery of useful packet systems, nonlinear obstruction theory, multilevel refinement, arithmetic congruences, quantum gravity, automorphy, or local or global Langlands reciprocity. **Recorded dependencies.** None declared.

Evidence. AST: `TEXTUAL_SEMANTIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Boundary metadata. `promote_out_of_scope_domain_claim`

Frozen source locator. Statement lines 338–341.

A.2 Source section 2: Typed operator grammars, realizations, and seed systems

4. Typed operator grammar

Stable ID. `PSC-I-D-TYPED-GRAMMAR`

Kind and regime. definition; algebraic, linear

Statement.

Let $Q = (Q_0, Q_1, s, t)$ be a quiver with finite vertex set. A typed operator grammar over $\Bbbk$ is a pair

$$\mathfrak{G} = (Q, I),$$

where $I \triangleleft \Bbbk Q$ is a two-sided ideal generated by $\Bbbk$-linear combinations of paths having common source and common target. Write

$$\Lambda_{\mathfrak{G}} := \Bbbk Q / I.$$

A realization of $\mathfrak{G}$ is a family $(X_v)_{v \in Q_0}$ and linear maps

$$X_a : X_{s(a)} \longrightarrow X_{t(a)} \qquad (a \in Q_1)$$

that annihilate every relation in I. A morphism $f : X \to Y$ of realizations is a family of linear maps $f_v : X_v \to Y_v$ satisfying

$$f_{t(a)} X_a = Y_a f_{s(a)} \qquad (a \in Q_1).$$

Recorded dependencies. None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 350–370.

5. Collapsed one-space realization

Stable ID. `PSC-I-D-COLLAPSED-REALIZATION`

Kind and regime. definition; linear

Statement.

For a typed realization X, define its collapsed one-space object by

$$|X| := \bigoplus_{v \in Q_0} X_v.$$

Every vertex idempotent acts as a coordinate projector on $|X|$, and every path acts as a source-target block operator. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `REALIZE_GRAMMAR` (`PLANNER_ONLY`)

Direct gate operations. `composition`, `identity`

Frozen source locator. Statement lines 373–380.

6. Typed realization and module equivalence

Stable ID. `PSC-I-T-TYPED-MODULE-EQUIVALENCE`

Kind and regime. theorem; algebraic

Statement.

The category of realizations of $\mathfrak{G}$ is equivalent to the category of unital left $\Lambda_{\mathfrak{G}}$-modules. Explicitly, a realization X determines the collapsed space $|X|$ of `PSC-I-D-COLLAPSED-REALIZATION` with the vertex idempotent e_v acting as the coordinate projector onto X_v and each arrow acting in its declared source-target block. Conversely, a unital $\Lambda_{\mathfrak{G}}$-module M determines the realization

$$M_v = e_v M, \qquad M_a(x) = ax.$$

The two constructions are inverse up to canonical natural isomorphism.

Attached source proof. Because Q_0 is finite, $1 = \sum_{v \in Q_0} e_v$ in $\Bbbk Q$, the e_v are pairwise orthogonal, and the same identities hold in $\Lambda_{\mathfrak{G}}$. Given a realization, define the action of a path by composition of its arrow maps and extend linearly. Every element of I acts as zero, so the action factors through $\Lambda_{\mathfrak{G}}$. The vertex idempotents give

$$|X| = \bigoplus_v e_v|X| = \bigoplus_v X_v.$$

Conversely, if M is unital, then

$$M = 1M = \sum_v e_v M.$$

Orthogonality makes the sum direct. For an arrow $a : v \to w$, the identities $a = e_w a e_v$ imply

$$a(e_v M) \subseteq e_w M.$$

The relations in I act as zero because M is a $\Lambda_{\mathfrak{G}}$-module. Applying both constructions returns the original vertex spaces and arrow maps, or the original module decomposition, through the evident identity maps. These identifications commute with morphisms, giving an equivalence of categories. □

Recorded dependencies. `PSC-I-D-TYPED-GRAMMAR`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `REALIZE_GRAMMAR` (`PLANNER_ONLY`)

Direct gate operations. `composition`, `identity`

Frozen source locator. Statement lines 383–393; proof lines 397–411.

7. Realized operator algebra

Stable ID. `PSC-I-D-REALIZED-ALGEBRA`

Kind and regime. definition; algebraic, linear

Statement.

For a realization X, the realized operator algebra is

$$\mathcal{A}_X := \operatorname{im}(\Lambda_{\mathfrak{G}} \longrightarrow \operatorname{End}_{\Bbbk}(|X|)) .$$

It records the actual operators in the realization and may be a proper quotient of the abstract grammar algebra. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 427–434.

8. Action factorization and hull invariance

Stable ID. `PSC-I-P-ACTION-FACTORIZATION`

Kind and regime. proposition; algebraic, linear

Statement.

Let $S \subseteq |X|$. The generated submodule under the grammar algebra equals the generated submodule under the realized operator algebra:

$$\Lambda_{\mathfrak{G}} S = \mathcal{A}_X S.$$

Consequently, every seed-rank statement depends only on the realized action, not on redundant relations in a chosen presentation of the grammar.

Attached source proof. The action map $\rho : \Lambda_{\mathfrak{G}} \twoheadrightarrow \mathcal{A}_X$ is surjective by definition and satisfies $\lambda s = \rho(\lambda) s$. Hence every grammar orbit vector lies in $\mathcal{A}_X S$. The reverse inclusion holds because every element of $\mathcal{A}_X$ is $\rho(\lambda)$ for some λ. □

Recorded dependencies. `PSC-I-D-REALIZED-ALGEBRA`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `FACTOR_ACTION` (`PLANNER_ONLY`)

Direct gate operations. `composition`

Frozen source locator. Statement lines 437–444; proof lines 448–450.

9. Linear seed system

Stable ID. `PSC-I-D-SEED-SYSTEM`

Kind and regime. definition; algebraic, linear, topological

Statement.

A linear seed system is a tuple

$$\mathfrak{S} = (X, \mathcal{A}, S, \mathcal{P}, \tau),$$

where X is a unital left $\mathcal{A}$-module, $\mathcal{A}$ is fixed independently of the target element, $S = (s_1, \ldots, s_r)$ is a finite distinguished seed family, $\mathcal{P}$ is either empty or a declared packet system, and τ is either the algebraic convention or a specified Hausdorff topology in which the declared operator action is continuous. **Recorded dependencies.** None declared.

Evidence. AST: `TYPED_CORE`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 458–465.

10. Generated hull

Stable ID. `PSC-I-D-GENERATED-HULL`

Kind and regime. definition; topological

Statement.

For $S \subseteq X$, define

$$\mathrm{Hull}_{\mathcal{A}}(S) := \sum_{s \in S} \mathcal{A}s.$$

In the topological regime define

$$\mathrm{Hull}_{\mathcal{A},\tau}(S) := \overline{\mathrm{Hull}_{\mathcal{A}}(S)}^{\tau}.$$

For a singleton $S = \{s\}$, write $\mathrm{Hull}_{\mathcal{A}}(s)$. **Recorded dependencies.** None declared.

Evidence. AST: `TYPED_CORE`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `BUILD_SEED_HULL` (`PLANNER_ONLY`)

Direct gate operations. `finite_sum`, `scalar_action`

Frozen source locator. Statement lines 468–479.

11. Natural presentation

Stable ID. `PSC-I-D-NATURAL-PRESENTATION`

Kind and regime. definition; linear, topological

Statement.

A seed presentation is natural relative to a declared grammar when the primitive operators and relations are fixed independently of the target, the relation system is finite or effectively enumerable, the topology is intrinsic to the realization, no primitive operator directly stores the complete target, and the presentation exposes a reusable invariant, theorem, or computation not encoded by a target-dependent primitive. **Recorded dependencies.** None declared.

Evidence. AST: `TEXTUAL_SEMANTIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 482–485.

12. Algebra generation and module generation are distinct

Stable ID. `PSC-I-P-ALGEBRA-MODULE-GENERATION`

Kind and regime. proposition; algebraic

Statement.

Every nonzero unital algebra A, regarded as a left module over itself, has

$$\mathrm{srank}_A(A) = 1,$$

with cyclic seed 1_A. This conclusion is independent of the number of algebra generators required to present A as an algebra.

Attached source proof. For every $a \in A$, one has $a = a1_A$. Hence $A1_A = A$. The number of operations needed to generate A as an algebra is a different invariant and does not enter the module calculation. □

Recorded dependencies. PSC-I-D-GENERATED-HULL

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Frozen source locator. Statement lines 488–495; proof lines 499–501.

13. Target-dependent operators

Stable ID. PSC-I-B-TARGET-DEPENDENT-OPERATORS

Kind and regime. boundary; linear

Statement.

For a fixed seed s and target x, the existence of a specially defined operator A_x satisfying $A_x s = x$ does not establish a natural seed calculus unless A_x belongs to a target-independent grammar satisfying PSC-I-D-NATURAL-PRESENTATION. **Recorded dependencies.** None declared.

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Boundary metadata. treat_target_dependent_operators_as_fixed_predeclared_grammar

Frozen source locator. Statement lines 503–506.

A.3 Source section 3: Free presentations by seeds and relations

14. Seed evaluation map

Stable ID. `PSC-I-D-SEED-EVALUATION`

Kind and regime. definition; algebraic, linear

Statement.

Let A be a unital ring, let M be a left A-module, and let $S = (s_1, \dots, s_r) \in M^r$. Define the A-linear map

$$\varepsilon_S : A^{\oplus r} \longrightarrow M, \qquad (a_1, \dots, a_r) \longmapsto \sum_{j=1}^{r} a_j s_j.$$

Recorded dependencies. None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `BUILD_SEED_HULL` (`EXACT_FIXTURE_ONLY`)

Direct gate operations. `finite_sum`, `scalar_action`

Frozen source locator. Statement lines 521–529.

15. Finite free presentation

Stable ID. `PSC-I-T-FREE-PRESENTATION`

Kind and regime. theorem; algebraic

Statement.

For the map in `PSC-I-D-SEED-EVALUATION`,

$$\operatorname{im} \varepsilon_S = \operatorname{Hull}_A(S), \qquad N_S := \operatorname{Ker} \varepsilon_S \leq A^{\oplus r},$$

and the induced map is an isomorphism of left A-modules

$$\boxed{A^{\oplus r}/N_S \cong \operatorname{Hull}_A(S).}$$

Consequently, S generates M if and only if ε_S is surjective.

Attached source proof. The image consists exactly of finite sums $\sum_j a_j s_j$, which is $\operatorname{Hull}_A(S)$. Because ε_S is A-linear, its kernel is a left submodule. The first isomorphism theorem gives

$$A^{\oplus r}/N_S \cong \operatorname{im} \varepsilon_S = \operatorname{Hull}_A(S).$$

The final equivalence is the equality $\operatorname{im} \varepsilon_S = M$. □

Recorded dependencies. `PSC-I-D-SEED-EVALUATION`

External dependencies. the first isomorphism theorem

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `PRESENT_AS_FREE_QUOTIENT` (`EXACT_FIXTURE_ONLY`)

Direct gate operations. `finite_sum`, `quotient`, `scalar_action`

Frozen source locator. Statement lines 532–545; proof lines 550–556.

16. Minimal free-cover characterization

Stable ID. `PSC-I-C-MINIMAL-FREE-COVER`

Kind and regime. corollary; algebraic

Statement.

For every finitely generated left A-module M,

$$\boxed{\operatorname{srank}_A(M) = \min\{r \geq 0 : \text{there exists a surjection } A^{\oplus r} \twoheadrightarrow M\}.}$$

Attached source proof. A seed family of size r gives a surjection by `PSC-I-T-FREE-PRESENTATION`. Conversely, the images of the standard basis vectors under any surjection $A^{\oplus r} \twoheadrightarrow M$ form a generating family of size at most r. □

Recorded dependencies. `PSC-I-T-FREE-PRESENTATION`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `PRESENT_AS_FREE_QUOTIENT` (`PLANNER_ONLY`)

Direct gate operations. `finite_sum`, `quotient`, `scalar_action`

Frozen source locator. Statement lines 573–581; proof lines 585–587.

17. One-seed presentation

Stable ID. `PSC-I-C-ONE-SEED-PRESENTATION`

Kind and regime. corollary; algebraic, linear

Statement.

For $s \in M$, the annihilator

$$\operatorname{Ann}_A(s) := \{a \in A : as = 0\}$$

is a left ideal and

$$\boxed{A/\operatorname{Ann}_A(s) \cong \operatorname{Hull}_A(s).}$$

Attached source proof. The one-seed evaluation map is $a \mapsto as$. Its kernel is $\operatorname{Ann}_A(s)$, which is therefore a left ideal, and `PSC-I-T-FREE-PRESENTATION` gives the quotient isomorphism. □

Recorded dependencies. `PSC-I-T-FREE-PRESENTATION`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `PRESENT_AS_FREE_QUOTIENT` (`PLANNER_ONLY`)

Direct gate operations. `finite_sum`, `quotient`, `scalar_action`

Frozen source locator. Statement lines 589–599; proof lines 603–605.

18. Two-sided annihilator criterion

Stable ID. `PSC-I-P-ANNIHILATOR-TWO-SIDED`

Kind and regime. proposition; algebraic, linear

Statement.

For $s \in M$, the left ideal $L = \mathrm{Ann}_A(s)$ is two-sided if and only if

$$L\ \mathrm{Hull}_A(s) = 0.$$

If s is cyclic, these conditions are equivalent to

$$\mathrm{Ann}_A(s) = \mathrm{Ann}_A(M).$$

Attached source proof. The left ideal L is two-sided exactly when $ab \in L$ for every $a \in L$ and $b \in A$. This is equivalent to

$$0 = (ab)s = a(bs) \qquad (a \in L,\ b \in A).$$

By additivity, the last identity is equivalent to $L\,\mathrm{Hull}_A(s) = 0$.

If s is cyclic, then $\mathrm{Hull}_A(s) = M$. Hence the preceding condition is $L \subseteq \mathrm{Ann}_A(M)$. The reverse inclusion always holds because an operator annihilating all of M annihilates s. Thus equality follows. Conversely, $\mathrm{Ann}_A(M)$ is a two-sided ideal, so equality implies that L is two-sided. □

Recorded dependencies. `PSC-I-C-ONE-SEED-PRESENTATION`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 608–618; proof lines 622–631.

19. Completion of a dense free presentation

Stable ID. `PSC-I-T-COMPLETED-PRESENTATION`

Kind and regime. theorem; algebraic, topological

Statement.

Let X be a complete Hausdorff locally convex space, let $\mathcal{A} \subseteq \mathrm{End}(X)$ act continuously, and let $S = (s_1, \dots, s_r)$. Put

$$E := \mathcal{A}^{\oplus r}/N_S$$

algebraically and let $j : E \to X$ be the injective map induced by ε_S. Give E the locally convex topology transported from $j(E)$, equivalently the topology generated by

$$\widetilde{p}(e) := p(j(e))$$

for the continuous seminorms p on X. Every $a \in \mathcal{A}$ acts continuously on E in the transported topology and therefore extends to $\widehat{E}$. If $\mathrm{Hull}_{\mathcal{A}}(S)$ is dense in X, then j extends uniquely to a topological $\mathcal{A}$-module isomorphism

$$\boxed{\widehat{E} \cong X.}$$

Attached source proof. By `PSC-I-T-FREE-PRESENTATION`, j is an algebraic $\mathcal{A}$-module isomorphism from E onto $\mathrm{Hull}_{\mathcal{A}}(S)$. The transported seminorms make it a topological isomorphism onto that subspace. For $a \in \mathcal{A}$ and a continuous seminorm p on X, continuity of $a : X \to X$ supplies continuous seminorms controlling $p(aj(e))$, so the induced action on E is continuous and extends uniquely to $\widehat{E}$. The subspace is dense by hypothesis. A Cauchy net in E is mapped to a Cauchy net

in X, which converges because X is complete. This defines a continuous extension $\widehat{j} : \widehat{E} \to X$. For any $x \in X$, choose a net (e_λ) in E with $j(e_\lambda) \to x$. The transported topology makes (e_λ) Cauchy, so it converges to some $\xi \in \widehat{E}$, and continuity gives $\widehat{j}(\xi) = x$. Thus $\widehat{j}$ is surjective. If $\widehat{j}(\xi) = 0$, represent ξ by a Cauchy net (e_λ). Then $j(e_\lambda) \to 0$, so every defining seminorm $\widetilde{p}(e_\lambda)$ tends to zero. Thus $e_\lambda \to 0$ in the completion and $\xi = 0$. Hence $\widehat{j}$ is bijective. Its inverse is continuous by the universal property of completion, or directly because the topology on E was transported from X. □

Recorded dependencies. `PSC-I-T-FREE-PRESENTATION`

External dependencies. uniqueness of Hausdorff completio

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 639–653; proof lines 657–659.

A.4 Source section 4: Seed rank and functorial calculus

20. Seed rank

Stable ID. PSC-I-D-SEED-RANK

Kind and regime. definition; algebraic, topological

Statement.

For a unital left A-module M, define

$$\operatorname{srank}_A(M) := \min\{|S| : AS = M\}.$$

For a topological module (X, τ), define

$$\operatorname{srank}_{\mathcal{A},\tau}(X) := \min\{|S| : \overline{\mathcal{A}S}^{\tau} = X\}.$$

Set $\operatorname{srank}_A(0) = 0$. When the value is finite, it is the minimal number of module generators.
Recorded dependencies. None declared.

Evidence. AST: TYPED_CORE. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPUTE_SEED_RANK (EXACT_FIXTURE_ONLY)

Frozen source locator. Statement lines 672–683.

21. Cyclic characterization

Stable ID. PSC-I-P-CYCLIC-CHARACTERIZATION

Kind and regime. proposition; algebraic, linear

Statement.

For $M \neq 0$,

$$\operatorname{srank}_A(M) = 1 \quad \Longleftrightarrow \quad M \text{ contains a cyclic vector.}$$

Attached source proof. Both conditions say that there is $m \in M$ with $Am = M$. □

Recorded dependencies. PSC-I-D-SEED-RANK

Evidence. AST: STRUCTURED_SYMBOLIC. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPUTE_SEED_RANK (EXACT_FIXTURE_ONLY)

Frozen source locator. Statement lines 686–694; proof lines 698–700.

22. Monotonicity under enlargement of the acting algebra

Stable ID. PSC-I-P-ACTING-ALGEBRA-MONOTONICITY

Kind and regime. proposition; algebraic, linear, topological

Statement.

Let $B \subseteq A \subseteq \operatorname{End}_{\mathbb{k}}(X)$ be unital operator algebras with the same identity. Then

$$\boxed{\operatorname{srank}_A(X) \leq \operatorname{srank}_B(X).}$$

The same statement holds for topological seed rank when both algebras act continuously.

Attached source proof. For every $S \subseteq X$, one has $BS \subseteq AS$. Hence every B-generating family is also an A-generating family. Taking closures gives the topological statement. □

Recorded dependencies. PSC-I-D-SEED-RANK

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Frozen source locator. Statement lines 702–709; proof lines 713–715.

23. Quotient monotonicity

Stable ID. PSC-I-T-QUOTIENT-MONOTONICITY

Kind and regime. theorem; algebraic, topological

Statement.

Let $\varphi : A \to B$ be a unital ring homomorphism and let

$$\Phi : M \twoheadrightarrow N$$

be a surjective additive map satisfying

$$\Phi(am) = \varphi(a)\Phi(m) \qquad (a \in A,\ m \in M).$$

Then

$$\boxed{\operatorname{srank}_B(N) \leq \operatorname{srank}_A(M).}$$

If M and N are topological modules, Φ is continuous and surjective, and the actions are continuous, then

$$\operatorname{srank}_{B,\tau_N}(N) \leq \operatorname{srank}_{A,\tau_M}(M).$$

Attached source proof. Let S generate M. For every $n \in N$, choose $m \in M$ with $\Phi(m) = n$ and write $m = \sum_i a_i s_i$. Then

$$n = \sum_i \varphi(a_i)\Phi(s_i),$$

so $\Phi(S)$ generates N.

For the topological statement, let S generate M densely. Given $n = \Phi(m)$, choose a net $m_\lambda \in AS$ converging to m. Continuity gives

$$\Phi(m_\lambda) \longrightarrow n,$$

and every $\Phi(m_\lambda)$ lies in $B\Phi(S)$. Thus $B\Phi(S)$ is dense in N. □

Recorded dependencies. PSC-I-D-GENERATED-HULL; PSC-I-D-SEED-RANK

Evidence. AST: STRUCTURED_SYMBOLIC. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. PUSH_GENERATORS_THROUGH_QUOTIENT (EXACT_FIXTURE_ONLY)

Direct gate operations. quotient

Frozen source locator. Statement lines 723–742; proof lines 747–759.

24. Isomorphism invariance

Stable ID. `PSC-I-C-ISOMORPHISM-INVARIANCE`

Kind and regime. corollary; algebraic, topological

Statement.

An intertwining module isomorphism preserves algebraic seed rank. A topological intertwining isomorphism preserves topological seed rank.

Attached source proof. Apply `PSC-I-T-QUOTIENT-MONOTONICITY` to the isomorphism and its inverse. □

Recorded dependencies. `PSC-I-T-QUOTIENT-MONOTONICITY`

Evidence. AST: `TEXTUAL_SEMANTIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `TRANSPORT_SEED_RANK` (`PLANNER_ONLY`)

Frozen source locator. Statement lines 766–769; proof lines 773–775.

25. Short exact sequence bounds

Stable ID. `PSC-I-T-EXACT-SEQUENCE-BOUNDS`

Kind and regime. theorem; algebraic

Statement.

Let

$$0 \longrightarrow L \xrightarrow{\iota} M \xrightarrow{\pi} N \longrightarrow 0$$

be a short exact sequence of left A-modules. Then

$$\boxed{\operatorname{srank}_A(N) \leq \operatorname{srank}_A(M) \leq \operatorname{srank}_A(L) + \operatorname{srank}_A(N).}$$

Attached source proof. The lower inequality is quotient monotonicity. For the upper inequality, choose generators $\ell_1, \ldots, \ell_r$ of L and generators $n_1, \ldots, n_s$ of N. Choose lifts $m_j \in M$ with $\pi(m_j) = n_j$. For $m \in M$, express

$$\pi(m) = \sum_{j=1}^{s} a_j n_j.$$

Then $m - \sum_j a_j m_j \in \iota(L)$ and is an A-linear combination of the $\iota(\ell_i)$. Hence the $r + s$ displayed vectors generate M. □

Recorded dependencies. `PSC-I-T-QUOTIENT-MONOTONICITY`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `COMBINE_SEED_SYSTEMS` (`EXACT_FIXTURE_ONLY`)

Direct gate operations. `direct_sum`, `tensor_product`

Frozen source locator. Statement lines 777–790; proof lines 794–800.

26. Shared direct-sum bounds

Stable ID. `PSC-I-P-SHARED-DIRECT-SUM`

Kind and regime. proposition; algebraic

Statement.

If one algebra A acts diagonally on $M \oplus N$, then

$$\boxed{\max\{\operatorname{srank}_A(M), \operatorname{srank}_A(N)\} \le \operatorname{srank}_A(M \oplus N) \le \operatorname{srank}_A(M) + \operatorname{srank}_A(N).}$$

Attached source proof. The coordinate projections are intertwining quotients and give the lower bound. For the upper bound, embed generators of M as $(m_i, 0)$ and generators of N as $(0, n_j)$. □

Recorded dependencies. `PSC-I-T-QUOTIENT-MONOTONICITY`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `COMBINE_SEED_SYSTEMS` (`PLANNER_ONLY`)

Direct gate operations. `direct_sum`, `tensor_product`

Frozen source locator. Statement lines 808–818; proof lines 822–824.

27. Independent direct-sum formula

Stable ID. `PSC-I-T-INDEPENDENT-DIRECT-SUM`

Kind and regime. theorem; algebraic

Statement.

Let M be a left A-module and N a left B-module. Under the product-algebra action

$$(a, b)(m, n) = (am, bn),$$

one has

$$\boxed{\operatorname{srank}_{A\times B}(M \oplus N) = \max\{\operatorname{srank}_A(M), \operatorname{srank}_B(N)\}.}$$

Attached source proof. The coordinate projections give the lower bound. Let r be the maximum on the right. Pad shorter generating lists by zeros and write them as $(m_1, \dots, m_r)$ and $(n_1, \dots, n_r)$. For arbitrary $m \in M$ and $n \in N$, choose $a_i \in A$ and $b_i \in B$ with

$$m = \sum_i a_i m_i, \qquad n = \sum_i b_i n_i.$$

Then

$$(m, n) = \sum_{i=1}^{r} (a_i, b_i)(m_i, n_i),$$

so the r paired vectors generate. □

Recorded dependencies. `PSC-I-T-QUOTIENT-MONOTONICITY`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `COMBINE_SEED_SYSTEMS` (`PLANNER_ONLY`)

Direct gate operations. `direct_sum`, `tensor_product`

Frozen source locator. Statement lines 826–840; proof lines 844–856.

28. Tensor-product bound

Stable ID. `PSC-I-P-TENSOR-BOUND`

Kind and regime. proposition; algebraic, linear

Statement.

Let A and B be unital $\Bbbk$-algebras, let M be a left A-module, and let N be a left B-module. Under the natural $A \otimes_\Bbbk B$ action,

$$\boxed{\operatorname{srank}_{A\otimes B}(M \otimes_\Bbbk N) \le \operatorname{srank}_A(M)\operatorname{srank}_B(N).}$$

Attached source proof. If $m_1, \dots, m_r$ generate M and $n_1, \dots, n_s$ generate N, then every simple tensor is a sum of vectors of the form

$$(am_i) \otimes (bn_j) = (a \otimes b)(m_i \otimes n_j).$$

Since simple tensors span the tensor product, the rs tensors $m_i \otimes n_j$ generate. □

Recorded dependencies. `PSC-I-D-SEED-RANK`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `COMBINE_SEED_SYSTEMS` (`PLANNER_ONLY`)

Direct gate operations. `direct_sum`, `tensor_product`

Frozen source locator. Statement lines 859–869; proof lines 873–879.

29. Tensor rank is not generally multiplicative

Stable ID. `PSC-I-B-TENSOR-NONEQUALITY`

Kind and regime. boundary; algebraic, linear

Statement.

The inequality in `PSC-I-P-TENSOR-BOUND` can be strict. Equality requires additional hypotheses on the acting algebras and their module multiplicities and is not exported as a universal law.
Recorded dependencies. None declared.

Evidence. AST: `TEXTUAL_SEMANTIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Boundary metadata. `infer_seed_rank_tensor_multiplicativity`

Frozen source locator. Statement lines 886–889.

30. Scalar-extension monotonicity

Stable ID. `PSC-I-P-SCALAR-EXTENSION-MONOTONICITY`

Kind and regime. proposition; algebraic

Statement.

Let $K/\Bbbk$ be a field extension, let A be a unital $\Bbbk$-algebra, and let M be a finitely generated left A-module. Set

$$A_K := K \otimes_\Bbbk A, \qquad M_K := K \otimes_\Bbbk M.$$

Then

$$\boxed{\operatorname{srank}_{A_K}(M_K) \le \operatorname{srank}_A(M).}$$

Every generating family $S \subseteq M$ extends to the generating family $1 \otimes S \subseteq M_K$.

Attached source proof. Let $S = \{s_1, \dots, s_r\}$ generate M. Every pure tensor $\lambda \otimes m \in M_K$ has an expression

$$m = \sum_{i=1}^{r} a_i s_i,$$

so

$$\lambda \otimes m = \sum_{i=1}^{r} (\lambda \otimes a_i)(1 \otimes s_i).$$

Pure tensors span M_K, hence $1 \otimes S$ generates M_K over A_K. Taking the minimum over S proves the inequality. □

Recorded dependencies. PSC-I-D-SEED-RANK

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. TRANSPORT_SEED_RANK (PLANNER_ONLY)

Frozen source locator. Statement lines 894–909; proof lines 914–926.

31. Seed-rank witness

Stable ID. PSC-I-D-SEED-RANK-WITNESS

Kind and regime. definition; algebraic

Statement.

A surjective intertwiner $\Phi : M \twoheadrightarrow N$ is a seed-rank witness of order r when $\operatorname{srank}(N) = r$. It certifies $\operatorname{srank}(M) \geq r$. **Recorded dependencies.** None declared.

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Frozen source locator. Statement lines 934–937.

32. Scalar quotient witness

Stable ID. PSC-I-P-SCALAR-QUOTIENT-WITNESS

Kind and regime. proposition; algebraic

Statement.

Let M be an A-module over a field $\Bbbk$. Suppose M has an r-dimensional quotient N on which A acts through a character

$$\chi : A \longrightarrow \Bbbk.$$

Then

$$\boxed{\operatorname{srank}_A(M) \geq r.}$$

Attached source proof. For every $n \in N$, the cyclic submodule An is contained in the one-dimensional space $\Bbbk n$. Therefore q seed images span a subspace of dimension at most q. Generating the r-dimensional quotient requires at least r seeds. Quotient monotonicity transfers the lower bound to M. □

Recorded dependencies. PSC-I-T-QUOTIENT-MONOTONICITY

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. PUSH_GENERATORS_THROUGH_QUOTIENT (PLANNER_ONLY)

Direct gate operations. quotient

Frozen source locator. Statement lines 940–950; proof lines 954–956.

A.5 Source section 5: Generator number modulo the Jacobson radical

33. Nakayama lemma

Stable ID. `PSC-I-CI-NAKAYAMA`

Kind and regime. classicalinput; algebraic

Statement.

Let A be a unital ring, let $J \subseteq \mathrm{Jac}(A)$, and let M be a finitely generated left A-module. If $M = N + JM$ for a submodule $N \leq M$, then $M = N$. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 969–972.

34. Generator number is detected modulo the Jacobson radical

Stable ID. `PSC-I-T-MODULE-TOP`

Kind and regime. theorem; algebraic

Statement.

Let A be a unital ring, let $J = \mathrm{Jac}(A)$, and let M be a finitely generated left A-module. A family $S \subseteq M$ generates M if and only if its image generates M/JM as an A/J-module. When A/J is semisimple Artinian, this quotient is the usual semisimple top; no semisimplicity of A/J is required for the general equality. Consequently,

$$\boxed{\mathrm{srank}_A(M) = \mathrm{srank}_{A/J}(M/JM).}$$

Attached source proof. If S generates M, its image generates every quotient. Conversely, suppose the image of S generates M/JM and put $N = AS$. Then

$$M = N + JM.$$

Nakayama's lemma gives $M = N$. Thus the same finite families generate before and after passage to the top, and the minimal cardinalities agree. □

Recorded dependencies. `PSC-I-CI-NAKAYAMA`; `PSC-I-T-QUOTIENT-MONOTONICITY`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `COMPUTE_SEED_RANK` (`PLANNER_ONLY`); `REDUCE_TO_SEMISIMPLE_TOP` (`PLANNER_ONLY`)

Direct gate operations. `jacobson_radical`, `quotient`, `semisimple_decomposition`

Frozen source locator. Statement lines 975–983; proof lines 988–994.

35. Local algebra formula

Stable ID. `PSC-I-C-LOCAL-ALGEBRA-FORMULA`

Kind and regime. corollary; algebraic

Statement.

Let A be a local ring with Jacobson radical J and division-ring residue algebra $D = A/J$. For every finitely generated left A-module M,

$$\boxed{\operatorname{srank}_A(M) = \dim_D(M/JM),}$$

where the dimension is the left D-dimension.

Attached source proof. The A/J-module M/JM is a left vector space over D. Its minimal number of generators is its D-dimension. □

Recorded dependencies. PSC-I-T-MODULE-TOP

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. REDUCE_TO_SEMISIMPLE_TOP (PLANNER_ONLY)

Direct gate operations. jacobson_radical, quotient, semisimple_decomposition

Frozen source locator. Statement lines 1009–1018; proof lines 1022–1024.

36. Wedderburn–Artin decomposition

Stable ID. PSC-I-CI-WEDDERBURN-ARTIN

Kind and regime. classicalinput; algebraic, linear

Statement.

Every semisimple Artinian ring is isomorphic to a finite product

$$A \cong \prod_{i=1}^{q} M_{d_i}(D_i)$$

of full matrix rings over division rings. **Recorded dependencies.** None declared.

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Frozen source locator. Statement lines 1027–1034.

37. Regular-module multiplicities

Stable ID. PSC-I-L-REGULAR-MODULE-MULTIPLICITIES

Kind and regime. lemma; algebraic

Statement.

Let

$$A = \prod_{i=1}^{q} M_{d_i}(D_i)$$

and let $S_i = D_i^{d_i}$ denote the natural simple left module for the ith factor, extended by zero across the other factors. Then, as a left A-module,

$$\boxed{A \cong \bigoplus_{i=1}^{q} S_i^{\oplus d_i}.}$$

Attached source proof. For one factor $M_d(D)$, let e_{jj} be the diagonal matrix units. The left regular module decomposes as

$$M_d(D) = \bigoplus_{j=1}^{d} M_d(D)e_{jj}.$$

Each minimal left ideal $M_d(D)e_{jj}$ is isomorphic to the natural column module D^d. Taking the direct product of factors gives the stated decomposition. □

Recorded dependencies. PSC-I-CI-WEDDERBURN-ARTIN

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Frozen source locator. Statement lines 1037–1047; proof lines 1051–1057.

38. Semisimple seed-rank formula over arbitrary division rings

Stable ID. PSC-I-T-SEMISIMPLE-SEED-RANK

Kind and regime. theorem; algebraic

Statement.

Let

$$A \cong \prod_{i=1}^{q} M_{d_i}(D_i)$$

be semisimple Artinian and let

$$M \cong \bigoplus_{i=1}^{q} S_i^{\oplus m_i}$$

be a finitely generated left A-module. Then

$$\boxed{\operatorname{srank}_A(M) = \max_{1\le i\le q} \left\lceil \frac{m_i}{d_i} \right\rceil .}$$

If A is any finite-dimensional algebra over a field, the same formula applied to the decomposition of $M/J(A)M$ over $A/J(A)$ computes $\operatorname{srank}_A(M)$.

Attached source proof. By PSC-I-C-MINIMAL-FREE-COVER, M is generated by r elements exactly when it is a quotient of $A^{\oplus r}$. By PSC-I-L-REGULAR-MODULE-MULTIPLICITIES,

$$A^{\oplus r} \cong \bigoplus_{i} S_i^{\oplus r d_i}.$$

Every short exact sequence of modules over a semisimple ring splits. Therefore a quotient isomorphic to M exists if and only if

$$m_i \le r d_i \qquad \text{for every } i.$$

The smallest such r is the displayed maximum. For a finite-dimensional nonsemisimple algebra, PSC-I-T-MODULE-TOP reduces the calculation to the semisimple module $M/J(A)M$ over $A/J(A)$. □

Recorded dependencies. PSC-I-T-MODULE-TOP; PSC-I-CI-WEDDERBURN-ARTIN; PSC-I-L-REGULAR-MODULE-MULTIPLICITIES; PSC-I-C-MINIMAL-FREE-COVER

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPUTE_SEED_RANK (PLANNER_ONLY); REDUCE_TO_SEMISIMPLE_TOP (PLANNER_ONLY)

Direct gate operations. jacobson_radical, quotient, semisimple_decomposition

Frozen source locator. Statement lines 1059–1079; proof lines 1084–1095.

39. Algebraically closed multiplicity formula

Stable ID. `PSC-I-C-ALGEBRAICALLY-CLOSED-FORMULA`

Kind and regime. corollary; algebraic

Statement.

If $\mathbb{k}$ is algebraically closed,

$$A \cong \prod_i M_{d_i}(\mathbb{k}), \qquad M/JM \cong \bigoplus_i (\mathbb{k}^{d_i})^{\oplus m_i},$$

then

$$\operatorname{srank}_A(M) = \max_i \left\lceil \frac{m_i}{d_i} \right\rceil.$$

Attached source proof. Take every $D_i = \mathbb{k}$ in `PSC-I-T-SEMISIMPLE-SEED-RANK`. □

Recorded dependencies. `PSC-I-T-SEMISIMPLE-SEED-RANK`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 1106–1118; proof lines 1122–1124.

40. Full matrix algebra

Stable ID. `PSC-I-C-FULL-MATRIX`

Kind and regime. corollary; algebraic, linear

Statement.

For the natural module $S = D^d$ over $M_d(D)$,

$$\operatorname{srank}_{M_d(D)}(S) = 1.$$

More strongly, every nonzero vector of S is cyclic.

Attached source proof. The formula gives $\lceil 1/d \rceil = 1$. Directly, let $v \neq 0$ and choose a coordinate $v_j \neq 0$. For any target $w = (w_i)$, the matrix whose jth column has entries $w_i v_j^{-1}$ and whose other columns vanish sends v to w. □

Recorded dependencies. `PSC-I-T-SEMISIMPLE-SEED-RANK`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 1126–1133; proof lines 1137–1139.

41. Multiplicity obstruction

Stable ID. `PSC-I-C-MULTIPLICITY-OBSTRUCTION`

Kind and regime. corollary; algebraic

Statement.

For the diagonal action of $M_d(D)$ on $S^{\oplus m}$,

$$\boxed{\operatorname{srank}_{M_d(D)}(S^{\oplus m}) = \left\lceil \frac{m}{d} \right\rceil.}$$

Attached source proof. Apply PSC-I-T-SEMISIMPLE-SEED-RANK to the single simple type with multiplicity m. □

Recorded dependencies. PSC-I-T-SEMISIMPLE-SEED-RANK

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Frozen source locator. Statement lines 1141–1151; proof lines 1155–1157.

A.6 Source section 6: Single-operator modules over arbitrary fields

42. Primary decomposition for polynomial modules

Stable ID. `PSC-I-CI-PID-PRIMARY-DECOMPOSITION`

Kind and regime. classicalinput; algebraic

Statement.

Every finite-dimensional $\Bbbk[t]$-module X decomposes as

$$X = \bigoplus_{p \in \mathrm{Irr}(\Bbbk[t])} X_p,$$

where only finitely many components are nonzero and every nonzero X_p is a finite direct sum of modules $\Bbbk[t]/(p^e)$. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 1170–1177.

43. Single-operator seed rank

Stable ID. `PSC-I-T-SINGLE-OPERATOR`

Kind and regime. theorem; algebraic, linear

Statement.

Let X be finite-dimensional over $\Bbbk$ and let $T \in \mathrm{End}_{\Bbbk}(X)$. Regard X as a $\Bbbk[t]$-module by $t \cdot x = Tx$. For every monic irreducible p, let X_p be the p-primary component and set

$$\mu_p(T) := \dim_{\Bbbk[t]/(p)} \big(X_p / p(T) X_p\big).$$

Then

$$\boxed{\mathrm{srank}_{\Bbbk[T]}(X) = \max_p \mu_p(T).}$$

Moreover, $\mu_p(T)$ is the number of cyclic p-primary summands in the elementary-divisor decomposition of X_p.

Attached source proof. Let the minimal polynomial be

$$m_T(t) = \prod_p p(t)^{e_p}.$$

The acting algebra is

$$\Bbbk[T] \cong \Bbbk[t]/(m_T) \cong \prod_p R_p, \qquad R_p := \Bbbk[t]/(p^{e_p}),$$

by the Chinese remainder theorem. The factor R_p acts on X_p and annihilates every other primary component. Hence the independent direct-sum formula gives

$$\mathrm{srank}_{\Bbbk[T]}(X) = \max_p \mathrm{srank}_{R_p}(X_p).$$

Each R_p is local with maximal ideal generated by the class of p and residue division field $K_p = \Bbbk[t]/(p)$. By `PSC-I-C-LOCAL-ALGEBRA-FORMULA`,

$$\mathrm{srank}_{R_p}(X_p) = \dim_{K_p} X_p / p(T) X_p = \mu_p(T).$$

Finally, every summand $\Bbbk[t]/(p^e)$ contributes one copy of K_p after quotienting by p, so the dimension equals the number of elementary-divisor summands. □

Recorded dependencies. PSC-I-CI-PID-PRIMARY-DECOMPOSITION; PSC-I-C-LOCAL-ALGEBRA-FORMULA; PSC-I-T-INDEPENDENT-DIRECT-SUM

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPUTE_SEED_RANK (PLANNER_ONLY); DECOMPOSE_PRIMARY (PLANNER_ONLY)

Direct gate operations. polynomial_action, primary_decomposition

Frozen source locator. Statement lines 1180–1194; proof lines 1201–1226.

44. Invariant-factor count

Stable ID. PSC-I-C-INVARIANT-FACTOR-COUNT

Kind and regime. corollary; algebraic

Statement.

If $X \neq 0$ and the rational canonical decomposition is written with nonconstant invariant factors

$$X \cong \bigoplus_{j=1}^{r} \mathbb{k}[t]/(f_j), \qquad f_1 \mid f_2 \mid \cdots \mid f_r,$$

then

$$\boxed{\operatorname{srank}_{\mathbb{k}[T]}(X) = r.}$$

Attached source proof. For each irreducible p, the number of invariant factors divisible by p equals the number of p-primary elementary divisors. Because $f_1 \mid \cdots \mid f_r$ and $f_1 \neq 1$ after zero summands are removed, some irreducible divisor of f_1 divides all r invariant factors. Hence $\max_p \mu_p(T) = r$. □

Recorded dependencies. PSC-I-T-SINGLE-OPERATOR

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPUTE_SEED_RANK (PLANNER_ONLY); DECOMPOSE_PRIMARY (PLANNER_ONLY)

Direct gate operations. polynomial_action, primary_decomposition

Frozen source locator. Statement lines 1238–1250; proof lines 1254–1256.

45. Geometric-multiplicity formula

Stable ID. PSC-I-C-ALGEBRAICALLY-CLOSED-SINGLE-OPERATOR

Kind and regime. corollary; algebraic, linear

Statement.

If $\mathbb{k}$ is algebraically closed, then

$$\boxed{\operatorname{srank}_{\mathbb{k}[T]}(X) = \max_{\lambda \in \operatorname{Spec}(T)} \dim_{\mathbb{k}} \operatorname{Ker}(T - \lambda I).}$$

Attached source proof. The irreducibles are $p = t - \lambda$. On the λ-primary component, the quotient by $(T - \lambda I)X_\lambda$ has dimension equal to the number of Jordan blocks, which is also $\dim \operatorname{Ker}(T - \lambda I)$. □

Recorded dependencies. PSC-I-T-SINGLE-OPERATOR

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPUTE_SEED_RANK (PLANNER_ONLY); DECOMPOSE_PRIMARY (PLANNER_ONLY)

Direct gate operations. polynomial_action, primary_decomposition

Frozen source locator. Statement lines 1258–1269; proof lines 1273–1275.

46. Cyclic-matrix criterion

Stable ID. PSC-I-C-CYCLIC-MATRIX

Kind and regime. corollary; algebraic, linear, topological

Statement.

For finite-dimensional X, the following are equivalent:

(i) T has a cyclic vector.

(ii) $\operatorname{srank}_{\Bbbk[T]}(X) = 1$.

(iii) Every p-primary component is cyclic.

(iv) The rational canonical form has one invariant factor.

(v) The minimal polynomial equals the characteristic polynomial.

(vi) After scalar extension to an algebraic closure, every eigenvalue has geometric multiplicity one.

Attached source proof. The equivalence of (i) and (ii) is PSC-I-P-CYCLIC-CHARACTERIZATION. By PSC-I-T-SINGLE-OPERATOR, (ii) holds exactly when every $\mu_p(T)$ equals one, which is (iii). Relatively prime primary cyclic modules combine into one cyclic module by the Chinese remainder theorem, so (iii) is equivalent to (iv). In rational canonical form, one invariant factor means that the unique invariant factor is both the minimal and characteristic polynomial, proving (iv)⇔(v). Scalar extension preserves the equality of minimal and characteristic polynomials. Over an algebraic closure, that equality is equivalent to one Jordan block for each eigenvalue, which is (vi). □

Recorded dependencies. PSC-I-T-SINGLE-OPERATOR; PSC-I-C-INVARIANT-FACTOR-COUNT; PSC-I-P-CYCLIC-CHARACTERIZATION

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. DECOMPOSE_PRIMARY (PLANNER_ONLY)

Direct gate operations. polynomial_action, primary_decomposition

Frozen source locator. Statement lines 1277–1288; proof lines 1292–1294.

A.7 Source section 7: Exact finite and countable packet coordinates

47. Finite packet system

Stable ID. `PSC-I-D-PACKET-SYSTEM`

Kind and regime. definition; linear

Statement.

A finite packet system on a $\Bbbk$-vector space X is a family

$$\mathcal{P} = (P_\alpha)_{\alpha \in I} \subseteq \mathrm{End}_\Bbbk(X)$$

satisfying

$$P_\alpha^2 = P_\alpha, \qquad P_\alpha P_\beta = 0 \quad (\alpha \neq \beta), \qquad \sum_{\alpha \in I} P_\alpha = I_X.$$

Set $X_\alpha = P_\alpha X$. Zero packet spaces may be removed without changing any identity. No orthogonality is assumed unless stated. **Recorded dependencies.** None declared.

Evidence. AST: `TYPED_CORE`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `STORED_KERNEL_EVIDENCE_MAPPED`.

Actions. `VALIDATE_PACKET_SYSTEM` (`RUNTIME_EXECUTABLE_EXACT_FINITE`)

Direct gate operations. `finite_sum`, `identity`, `packet_idempotents`

Frozen source locator. Statement lines 1307–1322.

48. Packet support and packet-pure vectors

Stable ID. `PSC-I-D-PACKET-SUPPORT`

Kind and regime. definition; linear

Statement.

For a finite packet system $\mathcal{P} = (P_\alpha)_{\alpha \in I}$ and $x \in X$, define

$$\mathrm{Supp}_{\mathcal{P}}(x) := \{\alpha \in I : P_\alpha x \neq 0\}.$$

The vector x is *packet-pure* if $|\mathrm{Supp}_{\mathcal{P}}(x)| \leq 1$. A finite family is packet-pure if each of its members is packet-pure, although different members may lie in different packets. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 1325–1334.

49. Finite packet aggregation

Stable ID. `PSC-I-L-PACKET-AGGREGATION`

Kind and regime. lemma; linear

Statement.

Let $\sigma : I \twoheadrightarrow J$ be a surjection and define

$$Q_j := \sum_{\sigma(\alpha) = j} P_\alpha.$$

Then $(Q_j)_{j\in J}$ is a finite packet system and

$$Q_j X = \bigoplus_{\sigma(\alpha)=j} X_\alpha.$$

Attached source proof. Pairwise annihilation of the P_α gives

$$Q_j^2 = Q_j, \qquad Q_j Q_k = 0 \quad (j \neq k).$$

Summing over j recovers $\sum_\alpha P_\alpha = I$. The range statement follows from the directness forced by pairwise annihilation. □

Recorded dependencies. `PSC-I-D-PACKET-SYSTEM`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `AGGREGATE_PACKETS` (`PLANNER_ONLY`)

Direct gate operations. `finite_sum`, `packet_idempotents`

Frozen source locator. Statement lines 1342–1352; proof lines 1356–1364.

50. Packet analysis and synthesis

Stable ID. `PSC-I-D-ANALYSIS-SYNTHESIS`

Kind and regime. definition; linear

Statement.

For a finite packet system, define

$$\mathcal{A}_{\mathcal{P}} : X \longrightarrow \bigoplus_{\alpha\in I} X_\alpha, \qquad x \longmapsto (P_\alpha x)_\alpha,$$

and

$$\mathcal{S}_{\mathcal{P}} : \bigoplus_{\alpha\in I} X_\alpha \longrightarrow X, \qquad (x_\alpha)_\alpha \longmapsto \sum_\alpha x_\alpha.$$

Recorded dependencies. None declared.

Evidence. AST: `TYPED_CORE`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `STORED_KERNEL_EVIDENCE_MAPPED`.

Actions. `ANALYZE_PACKETS` (`RUNTIME_EXECUTABLE_EXACT_FINITE`)

Direct gate operations. `direct_sum`, `finite_sum`, `identity`, `packet_idempotents`

Frozen source locator. Statement lines 1366–1380.

51. Exact packet-coordinate isomorphism

Stable ID. `PSC-I-T-EXACT-PACKET-COORDINATES`

Kind and regime. theorem; linear

Statement.

For every finite packet system,

$$\boxed{\mathcal{S}_{\mathcal{P}}\mathcal{A}_{\mathcal{P}} = I_X, \qquad \mathcal{A}_{\mathcal{P}}\mathcal{S}_{\mathcal{P}} = I_{\oplus X_\alpha}.}$$

Hence

$$\boxed{X \cong \bigoplus_{\alpha\in I} X_\alpha.}$$

Attached source proof. For $x \in X$,

$$\mathcal{S}_{\mathcal{P}}\mathcal{A}_{\mathcal{P}}x = \sum_{\alpha} P_\alpha x = x.$$

If $x_\alpha \in X_\alpha$, then $P_\beta x_\alpha = \delta_{\alpha\beta} x_\alpha$, so

$$\mathcal{A}_{\mathcal{P}}\mathcal{S}_{\mathcal{P}}(x_\alpha)_\alpha = \left(P_\beta \sum_{\alpha} x_\alpha\right)_\beta = (x_\beta)_\beta.$$

Thus analysis and synthesis are inverse linear maps. □

Recorded dependencies. PSC-I-D-PACKET-SYSTEM; PSC-I-D-ANALYSIS-SYNTHESIS

Evidence. AST: TYPED_CORE. Python: EXACT_FINITE_FIXTURE_PASS. Lean: STORED_KERNEL_EVIDENCE_MAPPED.

Actions. ANALYZE_PACKETS (RUNTIME_EXECUTABLE_EXACT_FINITE)

Direct gate operations. direct_sum, finite_sum, identity, packet_idempotents

Frozen source locator. Statement lines 1383–1397; proof lines 1402–1418.

52. Packetwise equality

Stable ID. PSC-I-C-PACKETWISE-EQUALITY

Kind and regime. corollary; linear

Statement.

For $x, y \in X$,

$$x = y \quad \Longleftrightarrow \quad P_\alpha x = P_\alpha y \text{ for every } \alpha.$$

Attached source proof. The forward implication is immediate. For the reverse implication, sum the packet equalities and use $\sum_\alpha P_\alpha = I$. □

Recorded dependencies. PSC-I-T-EXACT-PACKET-COORDINATES

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: STORED_KERNEL_EVIDENCE_MAPPED.

Actions. ANALYZE_PACKETS (RUNTIME_EXECUTABLE_EXACT_FINITE)

Direct gate operations. direct_sum, finite_sum, identity, packet_idempotents

Frozen source locator. Statement lines 1425–1433; proof lines 1437–1439.

53. Universal mapping property of finite packet coordinates

Stable ID. PSC-I-C-PACKET-UNIVERSAL-PROPERTY

Kind and regime. corollary; linear

Statement.

Let $\mathcal{P} = (P_\alpha)_{\alpha \in I}$ be a finite packet system on X, and let Y be a $\Bbbk$-vector space.

1. Every family $f_\alpha : X_\alpha \to Y$ determines a unique linear map $f : X \to Y$ with $f|_{X_\alpha} = f_\alpha$, namely
$$f = \sum_{\alpha} f_\alpha P_\alpha.$$

2. Every family $g_\alpha : Y \to X_\alpha$ determines a unique linear map $g : Y \to X$ with $P_\alpha g = g_\alpha$, namely

$$g = \sum_\alpha g_\alpha.$$

Equivalently,

$$\mathrm{Hom}(X, Y) \cong \prod_\alpha \mathrm{Hom}(X_\alpha, Y), \qquad \mathrm{Hom}(Y, X) \cong \prod_\alpha \mathrm{Hom}(Y, X_\alpha),$$

where the products are finite.

Attached source proof. For $x = \sum_\alpha P_\alpha x$, any extension of the f_α must satisfy

$$f(x) = \sum_\alpha f_\alpha(P_\alpha x),$$

which proves both existence and uniqueness of the first formula. For the second, the sum $g = \sum_\alpha g_\alpha$ satisfies

$$P_\beta g = \sum_\alpha P_\beta g_\alpha = g_\beta$$

because $g_\alpha(Y) \subseteq X_\alpha$. Uniqueness follows from $\sum_\alpha P_\alpha = I$. □

Recorded dependencies. `PSC-I-T-EXACT-PACKET-COORDINATES`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `ANALYZE_PACKETS` (`RUNTIME_EXECUTABLE_EXACT_FINITE`)

Direct gate operations. `direct_sum`, `finite_sum`, `identity`, `packet_idempotents`

Frozen source locator. Statement lines 1443–1463; proof lines 1468–1480.

54. Tensor-product packet systems

Stable ID. `PSC-I-T-TENSOR-PACKET-SYSTEM`

Kind and regime. theorem; algebraic, linear

Statement.

Let $\mathcal{P} = (P_\alpha)_{\alpha \in I}$ be a finite packet system on X, and let $\mathcal{Q} = (Q_\gamma)_{\gamma \in J}$ be a finite packet system on Y. Then

$$\mathcal{P} \boxtimes \mathcal{Q} := (P_\alpha \otimes Q_\gamma)_{(\alpha,\gamma) \in I \times J}$$

is a finite packet system on the algebraic tensor product $X \otimes_{\mathbb{k}} Y$, with packet spaces

$$(P_\alpha \otimes Q_\gamma)(X \otimes Y) = X_\alpha \otimes Y_\gamma.$$

Hence

$$\boxed{X \otimes Y \cong \bigoplus_{(\alpha,\gamma) \in I \times J} X_\alpha \otimes Y_\gamma.}$$

Attached source proof. For every pair of packet indices,

$$(P_\alpha \otimes Q_\gamma)^2 = P_\alpha^2 \otimes Q_\gamma^2 = P_\alpha \otimes Q_\gamma.$$

If $(\alpha, \gamma) \neq (\beta, \delta)$, then either $P_\alpha P_\beta = 0$ or $Q_\gamma Q_\delta = 0$, so the corresponding tensor idempotents annihilate each other. Finally,

$$\sum_{\alpha,\gamma} P_\alpha \otimes Q_\gamma = \left(\sum_\alpha P_\alpha\right) \otimes \left(\sum_\gamma Q_\gamma\right) = I_X \otimes I_Y.$$

The range is generated by pure tensors $P_\alpha x \otimes Q_\gamma y$, hence equals $X_\alpha \otimes Y_\gamma$. Exact reconstruction follows from PSC-I-T-EXACT-PACKET-COORDINATES. □

Recorded dependencies. PSC-I-D-PACKET-SYSTEM; PSC-I-T-EXACT-PACKET-COORDINATES

Evidence. AST: STRUCTURED_SYMBOLIC. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. TENSOR_PACKETS (EXACT_FIXTURE_ONLY)

Direct gate operations. finite_sum, packet_idempotents, tensor_product

Frozen source locator. Statement lines 1485–1507; proof lines 1512–1529.

55. Dual packet systems

Stable ID. PSC-I-P-DUAL-PACKET-SYSTEM

Kind and regime. proposition; algebraic, linear

Statement.

Let X be finite dimensional and let $\mathcal{P} = (P_\alpha)$ be a finite packet system. The algebraic dual maps

$$P_\alpha^* : X^* \to X^*, \qquad P_\alpha^*(\varphi) = \varphi \circ P_\alpha,$$

form a finite packet system on X^*. Its α-packet is canonically isomorphic to X_α^*, and

$$X^* \cong \bigoplus_\alpha X_\alpha^*.$$

Attached source proof. Because dualization reverses products,

$$P_\alpha^* P_\beta^* = (P_\beta P_\alpha)^* = \delta_{\alpha\beta} P_\alpha^*.$$

Also $\sum_\alpha P_\alpha^* = I_{X^*}$. Thus the dual maps form a packet system. Restriction gives a map from $P_\alpha^* X^*$ to X_α^*. It is injective because a covector in the range of P_α^* vanishes on every other packet, and it is surjective because any covector on X_α extends to X through the direct-sum decomposition. □

Recorded dependencies. PSC-I-D-PACKET-SYSTEM; PSC-I-T-EXACT-PACKET-COORDINATES

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. DUALIZE_PACKETS (PLANNER_ONLY)

Direct gate operations. dual, packet_idempotents

Frozen source locator. Statement lines 1531–1543; proof lines 1548–1555.

56. Finite Banach packet isomorphism

Stable ID. PSC-I-T-BANACH-PACKET-ISOMORPHISM

Kind and regime. theorem; linear, topological

Statement.

Let X be Banach and let $(P_\alpha)_{\alpha \in I}$ be a finite family of bounded pairwise annihilating idempotents with sum I_X. Then every range $X_\alpha = P_\alpha X$ is closed. For any finite product norm equivalent to the maximum norm on $\bigoplus_\alpha X_\alpha$, packet analysis and synthesis are bounded inverse operators.

Attached source proof. The range of a bounded idempotent is its fixed-point space,

$$P_\alpha X = \operatorname{Ker}(I - P_\alpha),$$

which is closed. Because I is finite, analysis and synthesis are bounded for the maximum and sum norms, respectively, and therefore for every equivalent finite product norm. Their inverse identities are algebraic and were proved in PSC-I-T-EXACT-PACKET-COORDINATES. □

Recorded dependencies. PSC-I-T-EXACT-PACKET-COORDINATES

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Frozen source locator. Statement lines 1562–1565; proof lines 1569–1575.

57. Packet-coordinate condition number

Stable ID. PSC-I-D-PACKET-CONDITION-NUMBER

Kind and regime. definition; linear, topological

Statement.

Fix a Banach direct-sum norm on $\bigoplus_\alpha X_\alpha$. Define

$$\kappa(\mathcal{P}) := \|\mathcal{A}_\mathcal{P}\|\,\|\mathcal{S}_\mathcal{P}\|.$$

Recorded dependencies. None declared.

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. EVALUATE_PACKET_CONDITIONING (PLANNER_ONLY)

Direct gate operations. norm, packet_idempotents

Frozen source locator. Statement lines 1577–1585.

58. Conditioned coordinate transport

Stable ID. PSC-I-P-CONDITIONED-COORDINATES

Kind and regime. proposition; linear, topological

Statement.

For a finite Banach packet system,

$$\kappa(\mathcal{P}) \geq 1.$$

If $T \in \mathcal{B}(X)$ and

$$[T]_\mathcal{P} := \mathcal{A}_\mathcal{P} T \mathcal{S}_\mathcal{P},$$

then

$$\boxed{\|[T]_\mathcal{P}\| \leq \kappa(\mathcal{P})\|T\|, \qquad \|T\| \leq \kappa(\mathcal{P})\|[T]_\mathcal{P}\|.}$$

Attached source proof. Since $I = \mathcal{S}_\mathcal{P}\mathcal{A}_\mathcal{P}$,

$$1 = \|I\| \leq \|\mathcal{S}_\mathcal{P}\|\|\mathcal{A}_\mathcal{P}\|.$$

The first displayed estimate follows from submultiplicativity. The inverse identity

$$T = \mathcal{S}_\mathcal{P}[T]_\mathcal{P}\mathcal{A}_\mathcal{P}$$

gives the second estimate. □

Recorded dependencies. PSC-I-T-BANACH-PACKET-ISOMORPHISM

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. EVALUATE_PACKET_CONDITIONING (PLANNER_ONLY)

Direct gate operations. norm, packet_idempotents

Frozen source locator. Statement lines 1588–1608; proof lines 1612–1622.

59. Completion preserves a finite packet split

Stable ID. PSC-I-C-COMPLETION-PACKET-SPLIT

Kind and regime. corollary; linear, topological

Statement.

Let M be a dense normed subspace of a Banach space X. Suppose every bounded P_α leaves M invariant and the restricted projectors form a packet system on M. Then

$$P_\alpha X = \overline{P_\alpha M}$$

and the packet decomposition of M extends uniquely to the completion X.

Attached source proof. For $x \in X$, choose $m_n \in M$ with $m_n \to x$. Continuity gives $P_\alpha m_n \to P_\alpha x$, so $P_\alpha X \subseteq \overline{P_\alpha M}$. The reverse inclusion follows because $P_\alpha X$ is closed and contains $P_\alpha M$. The bounded extensions remain pairwise annihilating idempotents whose finite sum is the identity. □

Recorded dependencies. PSC-I-T-BANACH-PACKET-ISOMORPHISM

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPLETE_PACKET_SPLIT (PLANNER_ONLY)

Direct gate operations. completion, packet_idempotents, topology

Frozen source locator. Statement lines 1629–1636; proof lines 1640–1642.

60. Unconditional packet sum space

Stable ID. PSC-I-D-UNCONDITIONAL-PACKET-SPACE

Kind and regime. definition; linear, topological

Statement.

Let $(X_n)_{n\ge 1}$ be closed subspaces of a real or complex Banach space X. Define

$$\bigoplus_{n\ge 1}^{u} X_n$$

to be the space of sequences $x = (x_n)$ with $x_n \in X_n$ for which $\sum_n x_n$ converges unconditionally in X. Equip it with

$$\|x\|_u := \sup_{F\in\mathbb{N}} \sup_{|a_n|\le 1} \left\| \sum_{n\in F} a_n x_n \right\|.$$

Recorded dependencies. None declared.

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Frozen source locator. Statement lines 1645–1659.

61. Completeness of the unconditional packet sum

Stable ID. `PSC-I-L-UNCONDITIONAL-COMPLETENESS`

Kind and regime. lemma; linear, topological

Statement.

The normed space $\bigoplus_{n\geq 1}^{u} X_n$ in `PSC-I-D-UNCONDITIONAL-PACKET-SPACE` is Banach.

Attached source proof. Let $x^{(k)} = (x_n^{(k)})$ be Cauchy in $\|\cdot\|_u$. For every fixed n,

$$\|x_n^{(k)} - x_n^{(\ell)}\| \leq \|x^{(k)} - x^{(\ell)}\|_u,$$

so $x_n^{(k)}$ converges to some $x_n \in X_n$ because X_n is closed. Given $\varepsilon > 0$, choose k_0 such that

$$\|x^{(k)} - x^{(\ell)}\|_u < \varepsilon \qquad (k, \ell \geq k_0).$$

For finite F and $|a_n| \leq 1$, pass $\ell \to \infty$ in the finite sum to obtain

$$\left\| \sum_{n\in F} a_n (x_n^{(k)} - x_n) \right\| \leq \varepsilon \qquad (k \geq k_0).$$

Hence $\|x^{(k)} - x\|_u \leq \varepsilon$ once x is known to be unconditionally summable.

Choose such a $k \geq k_0$. Since $x^{(k)}$ is unconditionally summable, there is a finite F_0 such that every finite $F \subseteq \mathbb{N} \setminus F_0$ and every $|a_n| \leq 1$ satisfy

$$\left\| \sum_{n\in F} a_n x_n^{(k)} \right\| < \varepsilon.$$

The preceding uniform estimate then gives

$$\left\| \sum_{n\in F} a_n x_n \right\| < 2\varepsilon.$$

Thus the tails of $\sum_n x_n$ are uniformly small under bounded scalar changes, which is the unconditional Cauchy criterion. Therefore $x \in \bigoplus^u X_n$, and the earlier estimate proves $x^{(k)} \to x$ in $\|\cdot\|_u$. □

Recorded dependencies. `PSC-I-D-UNCONDITIONAL-PACKET-SPACE`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 1662–1665; proof lines 1669–1698.

62. Countable unconditional packet isomorphism

Stable ID. `PSC-I-T-COUNTABLE-UNCONDITIONAL-PACKETS`

Kind and regime. theorem; linear, topological

Statement.

Let X be a real or complex Banach space and let $(P_n)_{n\geq 1} \subseteq \mathcal{B}(X)$ satisfy

$$P_n P_m = \delta_{nm} P_n$$

and

$$x = \sum_{n\geq 1} P_n x$$

unconditionally for every $x \in X$. Assume

$$K := \sup_{F\in\mathbb{N}} \sup_{|a_n|\le 1} \left\| \sum_{n\in F} a_n P_n \right\| < \infty.$$

Then analysis and synthesis define inverse Banach-space isomorphisms

$$\boxed{X \cong \bigoplus_{n\ge 1}^{u} P_n X.}$$

More precisely,

$$\|\mathcal{A}_{\mathcal{P}}\| \le K, \qquad \|\mathcal{S}_{\mathcal{P}}\| \le 1.$$

Attached source proof. Each $P_n X = \operatorname{Ker}(I - P_n)$ is closed. For $x \in X$,

$$\|\mathcal{A}_{\mathcal{P}} x\|_u = \sup_{F,a} \left\| \sum_{n\in F} a_n P_n x \right\| \le K\|x\|.$$

For $y = (y_n) \in \bigoplus^u P_n X$, define

$$\mathcal{S}_{\mathcal{P}} y = \sum_n y_n.$$

Since the all-one finite partial sums are among those used in $\|y\|_u$,

$$\|\mathcal{S}_{\mathcal{P}} y\| \le \|y\|_u.$$

The assumed expansion gives $\mathcal{S}_{\mathcal{P}}\mathcal{A}_{\mathcal{P}} x = x$. Conversely, boundedness of P_m permits termwise application to the unconditionally convergent series:

$$P_m \mathcal{S}_{\mathcal{P}} y = \sum_n P_m y_n = y_m.$$

Thus $\mathcal{A}_{\mathcal{P}}\mathcal{S}_{\mathcal{P}} = I$. Completeness of the coordinate space is PSC-I-L-UNCONDITIONAL-COMPLETENESS. □

Recorded dependencies. PSC-I-D-UNCONDITIONAL-PACKET-SPACE; PSC-I-L-UNCONDITIONAL-COMPLETENESS

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPLETE_PACKET_SPLIT (PLANNER_ONLY)

Direct gate operations. completion, packet_idempotents, topology

Frozen source locator. Statement lines 1700–1729; proof lines 1734–1760.

63. Uniform boundedness

Stable ID. PSC-I-R-UNIFORM-BOUNDEDNESS

Kind and regime. remark; linear, topological

Statement.

If every expansion $\sum_n P_n x$ is unconditional, the family of signed finite partial-sum operators is pointwise bounded. The uniform boundedness principle therefore yields a finite constant K. It is retained explicitly in PSC-I-T-COUNTABLE-UNCONDITIONAL-PACKETS because it is the quantitative coordinate bound exported downstream. **Recorded dependencies.** None declared.

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Frozen source locator. Statement lines 1762–1765.

64. Infinite packet sums

Stable ID. `PSC-I-B-INFINITE-PACKET-TOPOLOGY`

Kind and regime. boundary; linear, topological

Statement.

No identity $\sum_\alpha P_\alpha = I$ is used for an infinite family unless the convergence mode and the corresponding uniform boundedness statement are declared. Pointwise, strong, norm, weak, and unconditional convergence are not interchangeable. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Boundary metadata. `use_infinite_packet_identity_without_declared_convergence_and_uniform_boundedness`

Frozen source locator. Statement lines 1768–1771.

A.8 Source section 8: Transfer-block algebra and support patterns

65. Transfer block

Stable ID. `PSC-I-D-TRANSFER-BLOCK`

Kind and regime. definition; linear

Statement.

For $T \in \mathrm{End}_{\Bbbk}(X)$ and a finite packet system $\mathcal{P}$, define

$$T_{\beta\alpha} := P_\beta T P_\alpha \in \mathrm{Hom}_{\Bbbk}(X_\alpha, X_\beta).$$

Recorded dependencies. None declared.

Evidence. AST: `TYPED_CORE`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `STORED_KERNEL_EVIDENCE_MAPPED`.

Actions. `BLOCKIFY_OPERATOR` (`RUNTIME_EXECUTABLE_EXACT_FINITE`)

Direct gate operations. `composition`, `finite_sum`, `identity`, `packet_idempotents`, `transfer_blocks`

Frozen source locator. Statement lines 1785–1792.

66. Exact operator reconstruction

Stable ID. `PSC-I-T-OPERATOR-RECONSTRUCTION`

Kind and regime. theorem; linear

Statement.

Every $T \in \mathrm{End}_{\Bbbk}(X)$ satisfies

$$\boxed{T = \sum_{\alpha,\beta \in I} T_{\beta\alpha}.}$$

Attached source proof. Insert the finite packet identity on both sides:

$$T = \left(\sum_\beta P_\beta\right) T \left(\sum_\alpha P_\alpha\right) = \sum_{\alpha,\beta} P_\beta T P_\alpha.$$

□

Recorded dependencies. `PSC-I-T-EXACT-PACKET-COORDINATES`

Evidence. AST: `TYPED_CORE`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `STORED_KERNEL_EVIDENCE_MAPPED`.

Actions. `BLOCKIFY_OPERATOR` (`RUNTIME_EXECUTABLE_EXACT_FINITE`)

Direct gate operations. `composition`, `finite_sum`, `identity`, `packet_idempotents`, `transfer_blocks`

Frozen source locator. Statement lines 1795–1801; proof lines 1805–1815.

67. Packet matrix multiplication

Stable ID. `PSC-I-T-BLOCK-MULTIPLICATION`

Kind and regime. theorem; linear

Statement.

For $S, T \in \mathrm{End}_{\Bbbk}(X)$,

$$\boxed{(ST)_{\gamma\alpha} = \sum_{\beta \in I} S_{\gamma\beta} T_{\beta\alpha}.}$$

Attached source proof. Insert $I = \sum_{\beta} P_{\beta}$ between S and T:

$$P_{\gamma} S T P_{\alpha} = \sum_{\beta} P_{\gamma} S P_{\beta} T P_{\alpha}.$$

□

Recorded dependencies. `PSC-I-D-PACKET-SYSTEM`; `PSC-I-D-TRANSFER-BLOCK`

Evidence. AST: `TYPED_CORE`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `STORED_KERNEL_EVIDENCE_MAPPED`.

Actions. `COMPOSE_TRANSFER_BLOCKS` (`RUNTIME_EXECUTABLE_EXACT_FINITE`)

Direct gate operations. `composition`, `finite_sum`, `packet_idempotents`, `transfer_blocks`

Frozen source locator. Statement lines 1822–1832; proof lines 1837–1845.

68. Packet powers and paths

Stable ID. `PSC-I-C-BLOCK-POWERS`

Kind and regime. corollary; linear

Statement.

For $m \geq 1$,

$$(T^{m})_{\beta_m \beta_0} = \sum_{\beta_1, \ldots, \beta_{m-1}} T_{\beta_m \beta_{m-1}} \cdots T_{\beta_1 \beta_0}.$$

The same formula holds for a product of nonidentical operators with the corresponding blocks in temporal order.

Attached source proof. Induct on m using `PSC-I-T-BLOCK-MULTIPLICATION`. The mixed-operator statement is the same induction with a different operator at each step. □

Recorded dependencies. `PSC-I-T-BLOCK-MULTIPLICATION`

Evidence. AST: `TYPED_CORE`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `EXPAND_OPERATOR_PATHS` (`EXACT_FIXTURE_ONLY`)

Direct gate operations. `composition`, `finite_sum`, `path_composition`, `transfer_blocks`

Frozen source locator. Statement lines 1852–1864; proof lines 1868–1870.

69. Adjoint reversal

Stable ID. `PSC-I-C-ADJOINT-REVERSAL`

Kind and regime. corollary; linear

Statement.

If X is Hilbert, every P_α is orthogonal, and $T \in \mathcal{B}(X)$, then

$$\boxed{(T_{\beta\alpha})^* = (T^*)_{\alpha\beta}.}$$

Attached source proof. Because $P_\alpha^* = P_\alpha$,

$$(P_\beta T P_\alpha)^* = P_\alpha T^* P_\beta.$$

□

Recorded dependencies. `PSC-I-D-TRANSFER-BLOCK`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 1872–1878; proof lines 1882–1887.

70. Tensor-product transfer blocks

Stable ID. `PSC-I-C-TENSOR-TRANSFER-BLOCKS`

Kind and regime. corollary; linear

Statement.

Let $S \in \mathrm{End}(X)$ and $T \in \mathrm{End}(Y)$, with packet systems $\mathcal{P}$ and $\mathcal{Q}$. Relative to the tensor-product packet system of `PSC-I-T-TENSOR-PACKET-SYSTEM`,

$$\boxed{(S \otimes T)_{(\beta,\delta),(\alpha,\gamma)} = S_{\beta\alpha} \otimes T_{\delta\gamma}.}$$

Attached source proof. By definition,

$$\begin{aligned}(S \otimes T)_{(\beta,\delta),(\alpha,\gamma)} &= (P_\beta \otimes Q_\delta)(S \otimes T)(P_\alpha \otimes Q_\gamma) \\ &= (P_\beta S P_\alpha) \otimes (Q_\delta T Q_\gamma) \\ &= S_{\beta\alpha} \otimes T_{\delta\gamma}.\end{aligned}$$

□

Recorded dependencies. `PSC-I-T-TENSOR-PACKET-SYSTEM`; `PSC-I-D-TRANSFER-BLOCK`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `COMPOSE_TRANSFER_BLOCKS` (`RUNTIME_EXECUTABLE_EXACT_FINITE`)

Direct gate operations. `composition`, `finite_sum`, `packet_idempotents`, `transfer_blocks`

Frozen source locator. Statement lines 1891–1901; proof lines 1906–1914.

71. Blockwise intertwining equations

Stable ID. `PSC-I-T-BLOCK-INTERTWINING`

Kind and regime. theorem; linear

Statement.

Let $T \in \mathrm{End}(X)$, $S \in \mathrm{End}(Y)$, let $\mathcal{P} = (P_\alpha)_{\alpha \in I}$ and $\mathcal{Q} = (Q_\gamma)_{\gamma \in J}$ be finite packet systems, and let $U : X \to Y$. Define

$$T_{\beta\alpha} = P_\beta T P_\alpha, \qquad S_{\gamma\delta} = Q_\gamma S Q_\delta, \qquad U_{\gamma\alpha} = Q_\gamma U P_\alpha.$$

Then

$$\boxed{UT = SU}$$

if and only if, for every $\gamma \in J$ and $\alpha \in I$,

$$\boxed{\sum_{\beta \in I} U_{\gamma\beta} T_{\beta\alpha} = \sum_{\delta \in J} S_{\gamma\delta} U_{\delta\alpha}.}$$

Attached source proof. Apply Q_γ on the left and P_α on the right. Inserting $I_X = \sum_\beta P_\beta$ gives

$$Q_\gamma U T P_\alpha = \sum_\beta Q_\gamma U P_\beta T P_\alpha = \sum_\beta U_{\gamma\beta} T_{\beta\alpha}.$$

Inserting $I_Y = \sum_\delta Q_\delta$ gives

$$Q_\gamma S U P_\alpha = \sum_\delta Q_\gamma S Q_\delta U P_\alpha = \sum_\delta S_{\gamma\delta} U_{\delta\alpha}.$$

Thus $UT = SU$ implies every block equation. Conversely, if every block equation holds, then all blocks of $UT - SU$ vanish. Exact operator reconstruction yields $UT - SU = 0$. □

Recorded dependencies. `PSC-I-T-OPERATOR-RECONSTRUCTION`; `PSC-I-T-BLOCK-MULTIPLICATION`

Evidence. AST: `TYPED_CORE`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `LOWER_INTERTWINER` (`RUNTIME_EXECUTABLE_EXACT_FINITE`)

Direct gate operations. `composition`, `finite_sum`, `packet_idempotents`, `transfer_blocks`

Frozen source locator. Statement lines 1916–1938; proof lines 1943–1961.

72. Full block-algebra representation

Stable ID. `PSC-I-T-BLOCK-ALGEBRA`

Kind and regime. theorem; algebraic, linear

Statement.

The map

$$\Theta_{\mathcal{P}} : \mathrm{End}_{\Bbbk}(X) \longrightarrow \prod_{\beta,\alpha \in I} \mathrm{Hom}_{\Bbbk}(X_\alpha, X_\beta), \qquad T \longmapsto (T_{\beta\alpha})_{\beta,\alpha},$$

is an algebra isomorphism when the codomain is equipped with operator-valued block-matrix multiplication.

Attached source proof. Injectivity follows from exact reconstruction. Given arbitrary maps $B_{\beta\alpha} : X_\alpha \to X_\beta$, define

$$T\left(\sum_\alpha x_\alpha\right) := \sum_{\alpha,\beta} B_{\beta\alpha} x_\alpha.$$

The packet decomposition is unique and the index set is finite, so this defines a linear operator whose blocks are the prescribed maps. Thus $\Theta_{\mathcal{P}}$ is surjective. Compatibility with multiplication is PSC-I-T-BLOCK-MULTIPLICATION. □

Recorded dependencies. PSC-I-T-OPERATOR-RECONSTRUCTION; PSC-I-T-BLOCK-MULTIPLICATION

Evidence. AST: TYPED_CORE. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. ASSEMBLE_BLOCK_ALGEBRA (PLANNER_ONLY)

Direct gate operations. composition, direct_sum, finite_sum, transfer_blocks

Frozen source locator. Statement lines 1968–1979; proof lines 1984–1992.

73. Propagation of packet support

Stable ID. PSC-I-C-PACKET-SUPPORT-PROPAGATION

Kind and regime. corollary; linear

Statement.

Let $T \in \mathrm{End}(X)$ and $x \in X$. Then

$$\boxed{\mathrm{Supp}_{\mathcal{P}}(Tx) \subseteq \{\beta : \exists \alpha \in \mathrm{Supp}_{\mathcal{P}}(x) \text{ with } T_{\beta\alpha} \neq 0\}\,.}$$

Equality holds whenever, for every target β in the displayed right-hand set, the active contributions

$$\{T_{\beta\alpha} P_\alpha x : \alpha \in \mathrm{Supp}_{\mathcal{P}}(x),\ T_{\beta\alpha} \neq 0\}$$

have nonzero sum. In particular, equality holds if at most one active source packet feeds each such β and that source coordinate does not lie in the kernel of the corresponding block.

Attached source proof. For every target packet β,

$$P_\beta T x = \sum_{\alpha \in \mathrm{Supp}_{\mathcal{P}}(x)} T_{\beta\alpha} P_\alpha x.$$

If every block in this sum is zero, then the β-coordinate vanishes. This proves the inclusion. Equality is exactly the assertion that the displayed active sum is nonzero for every target in the right-hand set. The final sufficient condition leaves at most one nonzero summand at each target, and kernel avoidance makes that summand nonzero. □

Recorded dependencies. PSC-I-D-PACKET-SUPPORT; PSC-I-T-OPERATOR-RECONSTRUCTION

Evidence. AST: STRUCTURED_SYMBOLIC. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. PROPAGATE_PACKET_SUPPORT (EXACT_FIXTURE_ONLY)

Direct gate operations. support_relation, transfer_blocks

Frozen source locator. Statement lines 1996–2013; proof lines 2018–2026.

74. Dimension count

Stable ID. `PSC-I-C-DIMENSION-COUNT`

Kind and regime. corollary; algebraic

Statement.

If X is finite-dimensional and $d_\alpha = \dim X_\alpha$, then

$$\dim \mathrm{End}_{\Bbbk}(X) = \sum_{\alpha,\beta} d_\alpha d_\beta = \left(\sum_{\alpha} d_\alpha \right)^2 .$$

Attached source proof. Take dimensions in the block-algebra isomorphism and use $\dim \mathrm{Hom}(X_\alpha, X_\beta) = d_\alpha d_\beta$. □

Recorded dependencies. `PSC-I-T-BLOCK-ALGEBRA`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `ASSEMBLE_BLOCK_ALGEBRA` (`PLANNER_ONLY`)

Direct gate operations. `composition`, `direct_sum`, `finite_sum`, `transfer_blocks`

Frozen source locator. Statement lines 2029–2039; proof lines 2043–2045.

75. Block-diagonal projection and packet commutant

Stable ID. `PSC-I-P-BLOCK-DIAGONAL-COMMUTANT`

Kind and regime. proposition; algebraic, linear

Statement.

Define

$$\Delta_{\mathcal{P}}(T) := \sum_{\alpha} P_\alpha T P_\alpha .$$

Then $\Delta_{\mathcal{P}}$ is a linear idempotent on $\mathrm{End}(X)$ whose range is the block-diagonal subalgebra. For $T \in \mathrm{End}(X)$, the following are equivalent:

(i) $TP_\alpha = P_\alpha T$ for every α.

(ii) $T_{\beta\alpha} = 0$ whenever $\beta \neq \alpha$.

(iii) $\Delta_{\mathcal{P}}(T) = T$.

Attached source proof. Pairwise annihilation gives $\Delta_{\mathcal{P}}^2 = \Delta_{\mathcal{P}}$ and identifies its range with diagonal block families. If T commutes with all P_α, then for $\beta \neq \alpha$,

$$P_\beta T P_\alpha = P_\beta P_\alpha T = 0.$$

Thus (i) implies (ii). Exact reconstruction shows (ii) implies (iii). If $T = \sum_\alpha P_\alpha T P_\alpha$, then

$$TP_\gamma = P_\gamma T P_\gamma = P_\gamma T,$$

so (iii) implies (i). □

Recorded dependencies. `PSC-I-T-OPERATOR-RECONSTRUCTION`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `EXTRACT_PACKET_COMMUTANT` (`PLANNER_ONLY`)

Direct gate operations. `composition`, `packet_idempotents`, `transfer_blocks`

Frozen source locator. Statement lines 2047–2059; proof lines 2064–2074.

76. Support relation and support space

Stable ID. `PSC-I-D-SUPPORT-RELATION`

Kind and regime. definition; algebraic

Statement.

For a relation $R \subseteq I \times I$, with (β, α) interpreted as an allowed transfer $\alpha \to \beta$, define

$$\mathfrak{M}_R := \{T \in \mathrm{End}(X) : T_{\beta\alpha} = 0 \text{ whenever } (\beta, \alpha) \notin R\}.$$

Recorded dependencies. None declared.

Evidence. AST: `TYPED_CORE`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 2082–2090.

77. Support-pattern algebra criterion

Stable ID. `PSC-I-T-SUPPORT-PATTERN-ALGEBRA`

Kind and regime. theorem; algebraic

Statement.

Assume $X_\alpha \neq 0$ for every $\alpha \in I$. Then:

(i) $I_X \in \mathfrak{M}_R$ if and only if R is reflexive.

(ii) $\mathfrak{M}_R\mathfrak{M}_R \subseteq \mathfrak{M}_R$ if and only if R is transitive.

(iii) $\mathfrak{M}_R$ is a unital subalgebra if and only if R is a preorder.

Attached source proof. The identity has diagonal blocks I_{X_α} and no off-diagonal blocks, so it belongs to $\mathfrak{M}_R$ exactly when $(\alpha, \alpha) \in R$ for every α.

Assume R is transitive. If a summand

$$S_{\gamma\beta}T_{\beta\alpha}$$

in the (γ, α) block of ST is nonzero, then (γ, β) and (β, α) lie in R. Transitivity gives $(\gamma, \alpha) \in R$, so every forbidden block of ST vanishes.

Conversely, suppose transitivity fails. Choose α, β, γ with

$$(\beta, \alpha) \in R, \qquad (\gamma, \beta) \in R, \qquad (\gamma, \alpha) \notin R.$$

Choose nonzero vectors $u \in X_\alpha$, $v \in X_\beta$, and $w \in X_\gamma$. Choose linear functionals $f \in X_\alpha^*$ and $g \in X_\beta^*$ with $f(u) = g(v) = 1$. Let T have the sole block $x \mapsto f(x)v$ from X_α to X_β, and let S have the sole block $y \mapsto g(y)w$ from X_β to X_γ. Then $S, T \in \mathfrak{M}_R$, but ST has a nonzero forbidden (γ, α) block. This proves necessity. Combining reflexivity and transitivity gives (iii). □

Recorded dependencies. `PSC-I-T-BLOCK-MULTIPLICATION`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `STORED_KERNEL_EVIDENCE_MAPPED`.

Actions. `BUILD_SUPPORT_ALGEBRA` (`PLANNER_ONLY`)

Direct gate operations. `composition`, `finite_sum`, `support_relation`, `transfer_blocks`

Frozen source locator. Statement lines 2093–2101; proof lines 2105–2123.

78. $*$-closed bounded support patterns

Stable ID. `PSC-I-C-SUPPORT-PATTERN-STAR`

Kind and regime. corollary; algebraic, linear

Statement.

Let X be Hilbert, let the packet projectors be orthogonal, assume $X_\alpha \neq 0$ for every $\alpha \in I$, and put

$$\mathfrak{M}_R^{\mathrm{b}} := \mathfrak{M}_R \cap \mathcal{B}(X).$$

Then $\mathfrak{M}_R^{\mathrm{b}}$ is closed under adjoints if and only if R is symmetric. Hence $\mathfrak{M}_R^{\mathrm{b}}$ is a unital $*$-subalgebra of $\mathcal{B}(X)$ if and only if R is an equivalence relation.

Attached source proof. Adjoint reversal sends an allowed (β, α) block to an (α, β) block. Symmetry is therefore sufficient. If symmetry fails, choose a nonzero rank-one map in an allowed nonsymmetric block. Its adjoint occupies the forbidden reverse block, proving necessity. A reflexive, transitive, symmetric relation is an equivalence relation. □

Recorded dependencies. `PSC-I-C-ADJOINT-REVERSAL`; `PSC-I-T-SUPPORT-PATTERN-ALGEBRA`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `BUILD_SUPPORT_ALGEBRA` (`PLANNER_ONLY`)

Direct gate operations. `composition`, `finite_sum`, `support_relation`, `transfer_blocks`

Frozen source locator. Statement lines 2125–2132; proof lines 2136–2138.

79. Preorder support algebra decomposition

Stable ID. `PSC-I-T-PREORDER-ALGEBRA-DECOMPOSITION`

Kind and regime. theorem; algebraic, linear

Statement.

Let R be a preorder on the finite set I. Define

$$\alpha \sim \beta \quad \Longleftrightarrow \quad (\beta, \alpha) \in R \text{ and } (\alpha, \beta) \in R.$$

Let $\overline{I} = I/\!\sim$, which carries the induced partial order, and put

$$X_C := \bigoplus_{\alpha \in C} X_\alpha.$$

Let $\mathfrak{N}_R \subseteq \mathfrak{M}_R$ consist of operators whose blocks vanish whenever source and target belong to the same equivalence class. Then:

(i) $\mathfrak{N}_R$ is a two-sided nilpotent ideal.

(ii) If h is the maximal number of strict steps in a chain of $\overline{I}$, then

$$\mathfrak{N}_R^{h+1} = 0.$$

(iii) The diagonal-class map induces an algebra isomorphism

$$\boxed{\mathfrak{M}_R/\mathfrak{N}_R \cong \prod_{C\in\overline{I}} \mathrm{End}_{\Bbbk}(X_C).}$$

Attached source proof. A nonzero block in $\mathfrak{N}_R$ moves from one equivalence class to a strictly larger class in the quotient partial order. The block multiplication law shows that products concatenate such moves. Therefore left or right multiplication by an element of $\mathfrak{M}_R$ preserves the property of having no same-class blocks, so $\mathfrak{N}_R$ is a two-sided ideal. A product of $h+1$ elements of $\mathfrak{N}_R$ would require a strict chain of $h+1$ steps in $\overline{I}$, which does not exist. This proves nilpotence.

Let $P_C = \sum_{\alpha\in C} P_\alpha$. Define

$$\rho : \mathfrak{M}_R \longrightarrow \prod_C \mathrm{End}(X_C), \qquad \rho(T) = (P_C T P_C)_C.$$

If an intermediate class D contributes to $P_C S T P_C$, then the preorder contains transfers $C \to D$ and $D \to C$, forcing $C = D$ in the quotient poset. Hence

$$P_C S T P_C = (P_C S P_C)(P_C T P_C),$$

so ρ is an algebra homomorphism. Every collection of endomorphisms of the X_C defines an allowed block-diagonal-class operator, so ρ is surjective. Its kernel is exactly $\mathfrak{N}_R$. The first isomorphism theorem gives the quotient formula. □

Recorded dependencies. PSC-I-T-BLOCK-MULTIPLICATION; PSC-I-T-SUPPORT-PATTERN-ALGEBRA

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. BUILD_SUPPORT_ALGEBRA (PLANNER_ONLY)

Direct gate operations. composition, finite_sum, support_relation, transfer_blocks

Frozen source locator. Statement lines 2140–2168; proof lines 2173–2187.

A.9 Source section 9: Transfer graphs and directed packet generation

80. Transfer graph of an algebra-generating family

Stable ID. `PSC-I-D-TRANSFER-GRAPH`

Kind and regime. definition; algebraic, linear

Statement.

Let $\mathcal{P} = (P_\alpha)_{\alpha \in I}$ be a finite packet system on X with every $P_\alpha \in \mathcal{A}$. Let $\mathcal{G} \subseteq \mathcal{A}$ satisfy

$$\mathcal{A} = \mathrm{alg}_{\Bbbk}(\{P_\alpha : \alpha \in I\} \cup \mathcal{G}),$$

where $\mathrm{alg}_{\Bbbk}(E)$ denotes the unital $\Bbbk$-subalgebra generated by E. The transfer graph $\Gamma(\mathcal{G}, \mathcal{P})$ has vertex set I and an arrow

$$\alpha \longrightarrow \beta$$

when

$$P_\beta G P_\alpha \neq 0$$

for at least one $G \in \mathcal{G}$. Reachability includes paths of length zero. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `BUILD_TRANSFER_GRAPH` (`PLANNER_ONLY`)

Direct gate operations. `support_relation`, `transfer_blocks`

Frozen source locator. Statement lines 2200–2215.

81. Reachability support of the generated algebra

Stable ID. `PSC-I-T-ALGEBRA-REACHABILITY-SUPPORT`

Kind and regime. theorem; algebraic

Statement.

For every $A \in \mathcal{A}$,

$$P_\beta A P_\alpha = 0 \qquad \text{whenever } \beta \notin \mathrm{Reach}(\alpha).$$

Equivalently,

$$\mathcal{A} \subseteq \mathfrak{M}_{R_\Gamma}, \qquad R_\Gamma := \{(\beta, \alpha) : \beta \in \mathrm{Reach}(\alpha)\}.$$

Attached source proof. It is enough to treat a word in the generators and packet projectors. Insert

$$I = \sum_{\gamma \in I} P_\gamma$$

between consecutive nonprojector generators. Every resulting (β, α) block is a finite sum of products

$$P_\beta G_m P_{\gamma_{m-1}} G_{m-1} \cdots P_{\gamma_1} G_1 P_\alpha.$$

A nonzero product requires every consecutive block to be nonzero, hence gives a directed path

$$\alpha \to \gamma_1 \to \cdots \to \gamma_{m-1} \to \beta.$$

Packet projectors only insert stationary steps. Thus a word has no block from α to an unreachable β. Linear combinations preserve the same zero pattern. □

Recorded dependencies. PSC-I-D-TRANSFER-GRAPH; PSC-I-C-BLOCK-POWERS

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPUTE_REACHABILITY (PLANNER_ONLY)

Direct gate operations. composition, path_composition, support_relation

Frozen source locator. Statement lines 2218–2231; proof lines 2235–2251.

82. Reachability obstruction for generated hulls

Stable ID. PSC-I-P-REACHABILITY-OBSTRUCTION

Kind and regime. proposition; linear

Statement.

Let $I_0 \subseteq I$ and let

$$S \subseteq \bigoplus_{\alpha \in I_0} X_\alpha.$$

Then

$$\boxed{\mathrm{Hull}_{\mathcal{A}}(S) \subseteq \bigoplus_{\beta \in \mathrm{Reach}(I_0)} X_\beta.}$$

In particular, no seed supported outside a source strongly connected component can generate a nonzero vector in that source component.

Attached source proof. For $s \in S$, write $s = \sum_{\alpha \in I_0} P_\alpha s$. If β is unreachable from I_0, then for every $A \in \mathcal{A}$,

$$P_\beta A s = \sum_{\alpha \in I_0} P_\beta A P_\alpha s = 0$$

by PSC-I-T-ALGEBRA-REACHABILITY-SUPPORT. The same holds for every finite linear combination of orbit vectors. □

Recorded dependencies. PSC-I-T-ALGEBRA-REACHABILITY-SUPPORT

Evidence. AST: STRUCTURED_SYMBOLIC. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPUTE_REACHABILITY (EXACT_FIXTURE_ONLY)

Direct gate operations. composition, path_composition, support_relation

Frozen source locator. Statement lines 2259–2274; proof lines 2278–2287.

83. Packet-pure seed rank

Stable ID. PSC-I-D-DIRECTED-SEED-RANK

Kind and regime. definition; linear

Statement.

A vector is packet-pure when it lies in one packet space X_α. Define

$$\mathrm{psrank}_{\mathcal{A},\mathcal{P}}(X) := \min\{|S| : S \text{ is packet-pure and } \mathcal{A}S = X\}.$$

For $C \subseteq I$, put

$$P_C := \sum_{\alpha \in C} P_\alpha, \qquad X_C := P_C X, \qquad \mathcal{A}_C := P_C \mathcal{A} P_C,$$

and define the local packet-pure rank

$$r_C := \mathrm{psrank}_{\mathcal{A}_C, \mathcal{P}|_C}(X_C).$$

Recorded dependencies. None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `COMPUTE_SEED_RANK` (`PLANNER_ONLY`); `COMPUTE_PACKET_PURE_RANK` (`PLANNER_ONLY`)

Direct gate operations. `packet_idempotents`, `path_composition`, `support_relation`

Frozen source locator. Statement lines 2290–2310.

84. Basic packet-pure rank bounds

Stable ID. `PSC-I-P-PACKET-PURE-RANK-BOUNDS`

Kind and regime. proposition; algebraic, linear

Statement.

For every packet-resolved A-module with $P_\alpha \in A$,

$$\boxed{\mathrm{srank}_A(X) \le \mathrm{psrank}_{A,\mathcal{P}}(X) \le \sum_{\alpha \in I} \mathrm{srank}_{P_\alpha A P_\alpha}(X_\alpha).}$$

The right-hand corner algebra is unital on X_α with identity P_α.

Attached source proof. Every packet-pure generating family is an unrestricted generating family, which gives the first inequality. For each α, choose a minimal generating family $S_\alpha \subseteq X_\alpha$ for the corner action $P_\alpha A P_\alpha$. The union $S = \bigcup_\alpha S_\alpha$ is packet-pure. Since every $x \in X$ has the unique decomposition $x = \sum_\alpha x_\alpha$ and every x_α lies in $(P_\alpha A P_\alpha) S_\alpha \subseteq AS$, the union generates X. Its cardinality is the displayed sum. □

Recorded dependencies. `PSC-I-D-SEED-RANK`; `PSC-I-D-DIRECTED-SEED-RANK`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `COMPUTE_PACKET_PURE_RANK` (`PLANNER_ONLY`)

Direct gate operations. `packet_idempotents`, `path_composition`, `support_relation`

Frozen source locator. Statement lines 2313–2327; proof lines 2331–2333.

85. Edge-surjective packet system

Stable ID. `PSC-I-D-EDGE-SURJECTIVE`

Kind and regime. definition; algebraic, linear

Statement.

The packet-resolved module is edge-surjective relative to $\Gamma(\mathcal{G}, \mathcal{P})$ when, for every graph edge $\alpha \to \beta$, there exists $B \in \mathcal{A}$ such that the pure block

$$P_\beta B P_\alpha : X_\alpha \twoheadrightarrow X_\beta$$

is surjective. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `VERIFY_PROPAGATION_HYPOTHESES` (`PLANNER_ONLY`)

Direct gate operations. `support_relation`, `transfer_blocks`

Frozen source locator. Statement lines 2335–2342.

86. Source-component formula for packet-pure seed rank

Stable ID. `PSC-I-T-SOURCE-COMPONENT-SEED-RANK`

Kind and regime. theorem; algebraic, linear

Statement.

Assume I is finite, every packet space is nonzero, and the packet-resolved module is edge-surjective. Let $\mathfrak{C}_0$ be the set of source strongly connected components of the condensation graph of $\Gamma(\mathcal{G}, \mathcal{P})$. Then

$$\boxed{\operatorname{psrank}_{\mathcal{A},\mathcal{P}}(X) = \sum_{C \in \mathfrak{C}_0} r_C.}$$

Attached source proof. Let S be a packet-pure generating family for X and fix a source component C. Seeds outside C cannot reach C by PSC-I-P-REACHABILITY-OBSTRUCTION. Moreover, for $s \in X_C$ and $A \in \mathcal{A}$,

$$P_C A s = (P_C A P_C) s \in \mathcal{A}_C s.$$

Thus the seeds of S lying in X_C must generate X_C under $\mathcal{A}_C$, and there are at least r_C of them. Summing over disjoint source components gives

$$\operatorname{psrank}_{\mathcal{A},\mathcal{P}}(X) \geq \sum_{C \in \mathfrak{C}_0} r_C.$$

For the reverse inequality, choose for each source component C a packet-pure family S_C of size r_C generating X_C under $\mathcal{A}_C$, and put $S = \bigcup_C S_C$. Then every source component lies in $\mathcal{A}S$. Traverse the finite condensation graph in a topological order. Suppose all components preceding a non-source component D have already been shown to lie in $\mathcal{A}S$. Choose an incoming condensation edge from such a component to D, represented by a graph edge $\alpha \to \beta$ with α already generated and $\beta \in D$. Edge-surjectivity gives a pure block

$$B_{\beta\alpha} : X_\alpha \twoheadrightarrow X_\beta.$$

Because $X_\alpha \subseteq \mathcal{A}S$ and $\mathcal{A}S$ is $\mathcal{A}$-invariant, surjectivity implies $X_\beta \subseteq \mathcal{A}S$. For any other vertex $\gamma \in D$, choose a directed path inside the strongly connected component from β to γ. Successive surjective edge blocks propagate the full packet space along the path, so $X_\gamma \subseteq \mathcal{A}S$. Hence $X_D \subseteq \mathcal{A}S$. Induction over the condensation graph gives $X = \mathcal{A}S$, proving the upper bound. □

Recorded dependencies. PSC-I-P-REACHABILITY-OBSTRUCTION; PSC-I-D-EDGE-SURJECTIVE

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `COMPUTE_SEED_RANK` (`EXACT_FIXTURE_ONLY`); `COMPUTE_PACKET_PURE_RANK` (`EXACT_FIXTURE_ONLY`)

Direct gate operations. `packet_idempotents`, `path_composition`, `support_relation`

Frozen source locator. Statement lines 2345–2355; proof lines 2360–2377.

87. Packet-propagating system

Stable ID. `PSC-I-D-PACKET-PROPAGATING`

Kind and regime. definition; algebraic, linear

Statement.

An edge-surjective packet system is packet-propagating when every packet space X_α is cyclic under its diagonal corner algebra

$$P_\alpha \mathcal{A} P_\alpha.$$

Recorded dependencies. None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `VERIFY_PROPAGATION_HYPOTHESES` (`PLANNER_ONLY`)

Direct gate operations. `support_relation`, `transfer_blocks`

Frozen source locator. Statement lines 2384–2390.

88. Unweighted source-component count

Stable ID. `PSC-I-C-SOURCE-COMPONENT-COUNT`

Kind and regime. corollary; algebraic, linear

Statement.

Assume the packet-resolved module is packet-propagating and $X_\alpha \neq 0$ for every $\alpha \in I$. If c_0 denotes the number of source strongly connected components, then

$$\boxed{\operatorname{psrank}_{\mathcal{A},\mathcal{P}}(X) = c_0.}$$

Attached source proof. Fix a source component C and a vertex $\alpha \in C$. Choose a cyclic vector $s \in X_\alpha$ for $P_\alpha \mathcal{A} P_\alpha$. It generates all of X_α. Strong connectivity and edge-surjectivity then propagate the whole packet space to every vertex of C. Thus $r_C = 1$ for every source component. Apply `PSC-I-T-SOURCE-COMPONENT-SEED-RANK`. □

Recorded dependencies. `PSC-I-T-SOURCE-COMPONENT-SEED-RANK`

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `COMPUTE_PACKET_PURE_RANK` (`EXACT_FIXTURE_ONLY`)

Direct gate operations. `packet_idempotents`, `path_composition`, `support_relation`

Frozen source locator. Statement lines 2393–2401; proof lines 2405–2407.

89. Mixed-support seeds can merge source requirements

Stable ID. `PSC-I-B-PACKET-PURE-BOUNDARY`

Kind and regime. boundary; algebraic, linear

Statement.

The source-component formulas concern packet-pure seeds. They do not give the unrestricted seed rank. For example, let

$$\mathcal{A} = \Bbbk \times \Bbbk, \qquad X = \Bbbk \oplus \Bbbk$$

with the coordinate action and the two coordinate packet projectors. The transfer graph has two source components and

$$\mathrm{psrank}_{\mathcal{A},\mathcal{P}}(X) = 2,$$

but the mixed vector $(1, 1)$ is cyclic, so

$$\mathrm{srank}_{\mathcal{A}}(X) = 1.$$

Recorded dependencies. None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Boundary metadata. `identify_packet_pure_seed_rank_with_unrestricted_seed_rank`

Frozen source locator. Statement lines 2409–2425.

A.10 Source section 10: Coordinate transport, morphisms, invariant exact sequences, and completion

90. Coordinate transport

Stable ID. `PSC-I-T-COORDINATE-TRANSPORT`

Kind and regime. theorem; algebraic, linear

Statement.

Let $U : X \to Y$ be a vector-space isomorphism and let $\mathcal{P} = (P_\alpha)$ be a packet system on X. Define

$$Q_\alpha := UP_\alpha U^{-1}.$$

Then $\mathcal{Q} = (Q_\alpha)$ is a packet system on Y. For $\widetilde{T} = UTU^{-1}$,

$$\boxed{Q_\beta \widetilde{T} Q_\alpha = U(P_\beta T P_\alpha)U^{-1}.}$$

Hence packet dimensions, zero patterns, and support preorders are invariant under coordinate transport. If a declared algebra-generating family $\mathcal{G}$ is transported to $U\mathcal{G}U^{-1}$, then its transfer graph and reachability relation are also preserved.

Attached source proof. Conjugation preserves products and sums, so

$$Q_\alpha Q_\beta = UP_\alpha P_\beta U^{-1} = \delta_{\alpha\beta} Q_\alpha, \qquad \sum_\alpha Q_\alpha = I_Y.$$

The block identity is direct multiplication:

$$Q_\beta \widetilde{T} Q_\alpha = UP_\beta T P_\alpha U^{-1}.$$

All listed invariants are determined by these transported blocks. □

Recorded dependencies. `PSC-I-D-PACKET-SYSTEM`; `PSC-I-D-TRANSFER-BLOCK`

Evidence. AST: `TYPED_CORE`. Python: `EXACT_FINITE_FIXTURE_PASS`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `CHANGE_PACKET_COORDINATES` (`EXACT_FIXTURE_ONLY`)

Direct gate operations. `composition`, `packet_idempotents`, `transfer_blocks`

Frozen source locator. Statement lines 2443–2458; proof lines 2462–2477.

91. Packet-compatible operator morphism

Stable ID. `PSC-I-D-PACKET-MORPHISM`

Kind and regime. definition; algebraic, linear

Statement.

Let $(X, \mathcal{A}, \mathcal{P})$ and $(Y, \mathcal{B}, \mathcal{Q})$ be packet-resolved operator modules with packet index sets I and J. A packet-compatible operator morphism is a triple

$$(\Phi, \varphi, \sigma)$$

consisting of a unital algebra homomorphism $\varphi : \mathcal{A} \to \mathcal{B}$, a linear map $\Phi : X \to Y$ satisfying

$$\Phi(Ax) = \varphi(A)\Phi(x),$$

and a map $\sigma : I \to J$ satisfying

$$Q_j \Phi = \Phi P_{\sigma^{-1}(j)}, \qquad P_{\sigma^{-1}(j)} := \sum_{\sigma(\alpha)=j} P_\alpha.$$

An empty fiber is interpreted as the zero projector. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `TRANSPORT_MORPHISM` (`PLANNER_ONLY`)

Direct gate operations. `composition`, `finite_sum`, `packet_idempotents`, `transfer_blocks`

Frozen source locator. Statement lines 2484–2504.

92. Morphism of seeded systems

Stable ID. `PSC-I-D-SEED-MORPHISM`

Kind and regime. definition; linear, topological

Statement.

A morphism

$$(X, \mathcal{A}, S, \mathcal{P}) \longrightarrow (Y, \mathcal{B}, S', \mathcal{Q})$$

is a packet-compatible operator morphism satisfying the seed condition

$$\Phi(S) \subseteq \mathrm{Hull}_{\mathcal{B}}(S').$$

In the topological category all maps and actions are required to be continuous and the seed condition uses the topological hull when declared. **Recorded dependencies.** None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `TRANSPORT_MORPHISM` (`PLANNER_ONLY`)

Direct gate operations. `composition`, `finite_sum`, `packet_idempotents`, `transfer_blocks`

Frozen source locator. Statement lines 2507–2518.

93. Naturality of generated hulls and transfer blocks

Stable ID. `PSC-I-T-NATURALITY`

Kind and regime. theorem; linear

Statement.

Let (Φ, φ, σ) be a packet-compatible operator morphism. For every subset $S \subseteq X$,

$$\boxed{\Phi(\mathrm{Hull}_{\mathcal{A}}(S)) = \mathrm{Hull}_{\varphi(\mathcal{A})}(\Phi(S)) \subseteq \mathrm{Hull}_{\mathcal{B}}(\Phi(S)).}$$

For $T \in \mathcal{A}$ and $i, j \in J$,

$$\boxed{Q_j \varphi(T) Q_i \Phi = \sum_{\substack{\sigma(\beta)=j \\ \sigma(\alpha)=i}} \Phi P_\beta T P_\alpha.}$$

If Φ is surjective and S generates X, then $\Phi(S)$ generates Y under $\varphi(\mathcal{A})$ and hence under $\mathcal{B}$. If, in addition, the morphism is seeded, then S' generates Y.

Attached source proof. Every element of $\Phi(\mathrm{Hull}_{\mathcal{A}}(S))$ is a finite sum of vectors

$$\Phi(As) = \varphi(A)\Phi(s),$$

so it belongs to $\mathrm{Hull}_{\varphi(\mathcal{A})}(\Phi(S))$. Conversely, every generator $\varphi(A)\Phi(s)$ equals $\Phi(As)$, giving equality. For the block formula, use intertwining and packet compatibility twice:

$$\begin{aligned} Q_j\varphi(T)Q_i\Phi &= Q_j\varphi(T)\Phi P_{\sigma^{-1}(i)} \\ &= Q_j\Phi T P_{\sigma^{-1}(i)} \\ &= \Phi P_{\sigma^{-1}(j)} T P_{\sigma^{-1}(i)} \\ &= \sum_{\substack{\sigma(\beta)=j \\ \sigma(\alpha)=i}} \Phi P_\beta T P_\alpha. \end{aligned}$$

If Φ is surjective and $\mathrm{Hull}_{\mathcal{A}}(S) = X$, the hull equality gives $Y = \mathrm{Hull}_{\varphi(\mathcal{A})}(\Phi(S))$. If $\Phi(S) \subseteq \mathrm{Hull}_{\mathcal{B}}(S')$, then $\mathrm{Hull}_{\mathcal{B}}(\Phi(S)) \subseteq \mathrm{Hull}_{\mathcal{B}}(S')$, so S' generates Y. □

Recorded dependencies. PSC-I-D-PACKET-MORPHISM; PSC-I-T-BLOCK-MULTIPLICATION

Evidence. AST: TYPED_CORE. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. TRANSPORT_MORPHISM (EXACT_FIXTURE_ONLY)

Direct gate operations. composition, finite_sum, packet_idempotents, transfer_blocks

Frozen source locator. Statement lines 2521–2543; proof lines 2548–2565.

94. Category of seeded packet systems

Stable ID. PSC-I-P-SEED-SYSTEM-CATEGORY

Kind and regime. proposition; algebraic, linear

Statement.

Packet-compatible operator morphisms compose, seeded morphisms compose, and identity morphisms satisfy both definitions. Thus packet-resolved operator modules form a category **PktMod** and seeded packet systems form a category **SeedSys**.

Attached source proof. Let $(\Phi, \varphi, \sigma) : (X, \mathcal{A}, \mathcal{P}) \to (Y, \mathcal{B}, \mathcal{Q})$ and $(\Psi, \psi, \tau) : (Y, \mathcal{B}, \mathcal{Q}) \to (Z, \mathcal{C}, \mathcal{R})$ be morphisms. Intertwining is preserved because

$$\Psi\Phi(Ax) = \Psi(\varphi(A)\Phi x) = (\psi \circ \varphi)(A)\Psi\Phi x.$$

For k in the target packet set,

$$\begin{aligned} R_k\Psi\Phi &= \Psi Q_{\tau^{-1}(k)}\Phi \\ &= \Psi\Phi \sum_{\tau(\sigma(\alpha))=k} P_\alpha. \end{aligned}$$

Thus the composite packet map is $\tau \circ \sigma$. If the morphisms are seeded, then

$$\Phi(S) \subseteq \mathrm{Hull}_{\mathcal{B}}(S'), \qquad \Psi(S') \subseteq \mathrm{Hull}_{\mathcal{C}}(S'').$$

By PSC-I-T-NATURALITY,

$$\Psi\Phi(S) \subseteq \Psi(\mathrm{Hull}_{\mathcal{B}}(S')) \subseteq \mathrm{Hull}_{\mathcal{C}}(\Psi(S')) \subseteq \mathrm{Hull}_{\mathcal{C}}(S'').$$

Identity maps satisfy the definitions with identity algebra and packet maps. □

Recorded dependencies. PSC-I-D-PACKET-MORPHISM; PSC-I-D-SEED-MORPHISM; PSC-I-T-NATURALITY

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. TRANSPORT_MORPHISM (PLANNER_ONLY)

Direct gate operations. composition, finite_sum, packet_idempotents, transfer_blocks

Frozen source locator. Statement lines 2572–2575; proof lines 2581–2614.

95. Packet exact sequence for an invariant subspace

Stable ID. PSC-I-T-PACKET-EXACT-SEQUENCE

Kind and regime. theorem; linear

Statement.

Let $K \leq X$ satisfy $P_\alpha K \subseteq K$ for every α. Define

$$K_\alpha := P_\alpha K = K \cap X_\alpha$$

and

$$\overline{P}_\alpha(x + K) := P_\alpha x + K.$$

Then:

(i) The nonzero restrictions $P_\alpha|_K$ form a packet system on K.

(ii) The nonzero $\overline{P}_\alpha$ form a packet system on X/K.

(iii) For every α there is a short exact sequence

$$0 \longrightarrow K_\alpha \longrightarrow X_\alpha \longrightarrow \overline{P}_\alpha(X/K) \longrightarrow 0,$$

and

$$\overline{P}_\alpha(X/K) \cong X_\alpha/(X_\alpha \cap K) \cong (X_\alpha + K)/K.$$

(iv) The full exact sequence decomposes packetwise as the direct sum of these short exact sequences.

Attached source proof. Restriction preserves idempotency, pairwise annihilation, and the identity sum on K. If $x \in K \cap X_\alpha$, then $x = P_\alpha x \in P_\alpha K$, proving $K_\alpha = K \cap X_\alpha$.

The quotient operator is well-defined because replacing x by $x + k$ changes $P_\alpha x$ by $P_\alpha k \in K$. The idempotent and annihilation identities descend, and the sum is the identity on X/K. The map $X_\alpha \to X/K$ has image $(X_\alpha + K)/K$ and kernel $X_\alpha \cap K$, giving the short exact sequence and the quotient isomorphisms. Exact packet coordinates identify the original sequence with the direct sum over α. □

Recorded dependencies. PSC-I-T-EXACT-PACKET-COORDINATES

Evidence. AST: TYPED_CORE. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. RESOLVE_PACKET_EXACT_SEQUENCE (EXACT_FIXTURE_ONLY)

Direct gate operations. direct_sum, exact_sequence, kernel, packet_idempotents, quotient

Frozen source locator. Statement lines 2617–2648; proof lines 2653–2657.

96. Packet descent through an invariant quotient

Stable ID. PSC-I-T-QUOTIENT-PACKET-DESCENT

Kind and regime. theorem; algebraic, linear

Statement.

Under the hypotheses of PSC-I-T-PACKET-EXACT-SEQUENCE, the operators

$$\overline{P}_\alpha(x+K) = P_\alpha x + K$$

are well-defined pairwise annihilating idempotents with sum $I_{X/K}$, and the quotient map satisfies

$$\boxed{\pi P_\alpha = \overline{P}_\alpha \pi.}$$

After zero packets are removed, they form the canonical quotient packet system.

Attached source proof. All assertions are the quotient part of PSC-I-T-PACKET-EXACT-SEQUENCE. The intertwining identity follows directly from the definition:

$$\overline{P}_\alpha \pi(x) = P_\alpha x + K = \pi P_\alpha(x).$$

□

Recorded dependencies. PSC-I-T-PACKET-EXACT-SEQUENCE

Evidence. AST: TYPED_CORE. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. DESCEND_PACKET_QUOTIENT (EXACT_FIXTURE_ONLY)

Direct gate operations. packet_idempotents, quotient, transfer_blocks

Frozen source locator. Statement lines 2664–2675; proof lines 2679–2684.

97. Restriction and quotient of transfer blocks

Stable ID. PSC-I-C-QUOTIENT-PACKET-DESCENT

Kind and regime. corollary; algebraic, linear

Statement.

Under the hypotheses of PSC-I-T-PACKET-EXACT-SEQUENCE, let $T \in \mathrm{End}(X)$ preserve K. Then every block $T_{\beta\alpha}$ maps K_α into K_β, and the induced quotient operator $\overline{T}$ satisfies

$$\boxed{\overline{T}_{\beta\alpha}\,\pi = \pi T_{\beta\alpha},}$$

where $\pi : X \twoheadrightarrow X/K$.

Attached source proof. If $x \in K_\alpha$, then $Tx \in K$, so

$$T_{\beta\alpha}x = P_\beta T x \in K \cap X_\beta = K_\beta.$$

On the quotient,

$$\overline{T}_{\beta\alpha}\pi = \overline{P}_\beta \overline{T P}_\alpha \pi = \pi P_\beta T P_\alpha.$$

□

Recorded dependencies. PSC-I-T-PACKET-EXACT-SEQUENCE

Evidence. AST: STRUCTURED_SYMBOLIC. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. DESCEND_PACKET_QUOTIENT (EXACT_FIXTURE_ONLY)

Direct gate operations. packet_idempotents, quotient, transfer_blocks

Frozen source locator. Statement lines 2686–2697; proof lines 2701–2714.

98. Topological packet exact sequence

Stable ID. PSC-I-C-TOPOLOGICAL-PACKET-EXACT-SEQUENCE

Kind and regime. corollary; algebraic, linear, topological

Statement.

If X is Banach, the P_α are bounded, and K is closed and packet invariant, then every packet sequence in PSC-I-T-PACKET-EXACT-SEQUENCE is a short exact sequence of Banach spaces with bounded maps, and the quotient packets are bounded.

Attached source proof. Closedness of K makes X/K Banach. The restricted projectors are bounded, and the quotient projectors are bounded because

$$\|\overline{P}_\alpha(x+K)\|_{X/K} \le \|P_\alpha\|\,\|x+K\|_{X/K}.$$

The packet spaces are closed ranges of bounded idempotents. □

Recorded dependencies. PSC-I-T-PACKET-EXACT-SEQUENCE; PSC-I-T-BANACH-PACKET-ISOMORPHISM

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. RESOLVE_PACKET_EXACT_SEQUENCE (PLANNER_ONLY); COMPLETE_TOPOLOGICAL_REALIZATION (PLANNER_ONLY)

Direct gate operations. closure, completion, composition, direct_sum, exact_sequence, kernel, packet_idempotents, quotient, topology

Frozen source locator. Statement lines 2716–2719; proof lines 2724–2732.

99. Dense realization theorem

Stable ID. PSC-I-T-DENSE-REALIZATION

Kind and regime. theorem; linear, topological

Statement.

Let X and Y be Hausdorff topological vector spaces, let $\Phi : X \to Y$ be continuous, surjective, and φ-intertwining, and let S topologically generate X. Then $\Phi(S)$ topologically generates Y. If finite continuous packet systems satisfy

$$Q_j\Phi = \Phi P_{\sigma^{-1}(j)},$$

then

$$\boxed{Q_jY = \Phi(P_{\sigma^{-1}(j)}X)}$$

for every j.

Attached source proof. Given $y = \Phi(x)$, choose a net $x_\lambda \in \mathrm{Hull}_{\mathcal{A}}(S)$ converging to x. Then

$$\Phi(x_\lambda) \to y,$$

and naturality places every $\Phi(x_\lambda)$ in $\mathrm{Hull}_{\mathcal{B}}(\Phi(S))$. Hence the latter hull is dense. For packets, one inclusion follows from compatibility. Conversely, if $y = Q_j y$, choose x with $\Phi(x) = y$. Then

$$y = Q_j \Phi(x) = \Phi P_{\sigma^{-1}(j)} x.$$

□

Recorded dependencies. PSC-I-T-NATURALITY

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPLETE_TOPOLOGICAL_REALIZATION (PLANNER_ONLY)

Direct gate operations. closure, completion, composition, topology

Frozen source locator. Statement lines 2735–2748; proof lines 2752–2761.

100. Quotient topology and completed realization

Stable ID. PSC-I-T-QUOTIENT-TOPOLOGY-COMPLETION

Kind and regime. theorem; algebraic, linear, topological

Statement.

Let $\Phi : X \twoheadrightarrow Y$ be a continuous linear quotient map between Hausdorff topological vector spaces, and put $K = \operatorname{Ker} \Phi$. Then the induced map

$$\widetilde{\Phi} : X/K \longrightarrow Y$$

is a topological isomorphism. If $D \subseteq X$ is a linear subspace whose image $\Phi(D)$ is dense in a complete space Y, then the completion of

$$D/(D \cap K)$$

in the topology transported from Y is canonically isomorphic to Y.

Attached source proof. The algebraic first isomorphism theorem gives a bijection $\widetilde{\Phi}$. By definition of the quotient topology, a set $U \subseteq X/K$ is open exactly when its preimage in X is open. Because Φ is a quotient map, this is equivalent to $\widetilde{\Phi}(U)$ being open in Y. Thus $\widetilde{\Phi}$ is a homeomorphism. The second statement applies the dense completion argument of PSC-I-T-COMPLETED-PRESENTATION to the injective map $D/(D \cap K) \to Y$. □

Recorded dependencies. PSC-I-T-COMPLETED-PRESENTATION

External dependencies. the definition of quotient topology, and uniqueness of completio

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. COMPLETE_TOPOLOGICAL_REALIZATION (PLANNER_ONLY)

Direct gate operations. closure, completion, composition, topology

Frozen source locator. Statement lines 2763–2774; proof lines 2778–2780.

101. Continuous surjections and quotient topology

Stable ID. PSC-I-B-TOPOLOGICAL-QUOTIENT

Kind and regime. boundary; algebraic, topological

Statement.

Continuity and surjectivity transport density as in PSC-I-T-DENSE-REALIZATION. They do not by themselves identify the given topology on Y with the quotient topology from X. The stronger conclusion in PSC-I-T-QUOTIENT-TOPOLOGY-COMPLETION requires the quotient-map hypothesis or an equivalent open-mapping condition. **Recorded dependencies.** None declared.

Evidence. AST: STRUCTURED_SYMBOLIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Boundary metadata. promote_continuous_surjection_to_topological_quotient_isomorphism_without_quotient_hypotheses

Frozen source locator. Statement lines 2782–2785.

A.11 Source section 11: Realization functors and semantic theorem transport

102. Realization functor and preservation profile

Stable ID. `PSC-I-D-REALIZATION-FUNCTOR`

Kind and regime. definition; algebraic, linear, topological

Statement.

Let $\mathbf{SeedSys}_{\mathrm{abs}}$ be an abstract category of seed systems and let $\mathbf{C}_D$ be a domain category. A realization functor is a functor

$$\mathsf{Ev}_D : \mathbf{SeedSys}_{\mathrm{abs}} \longrightarrow \mathbf{C}_D$$

with a declared preservation profile specifying which of the following structures are preserved:

$$0,\ I,\ \text{finite sums},\ \text{composition},\ \text{biproducts},$$
$$\text{kernels},\ \text{cokernels},\ \text{quotients},\ \text{tensor products},$$
$$\text{packet idempotents},\ \text{transfer blocks},\ \text{topological completions}.$$

Preservation of a structure means preservation of its defining identity or universal property, not merely a similarity of notation. **Recorded dependencies.** None declared.

Evidence. AST: `TYPED_CORE`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `CHECK_REALIZATION_PROFILE` (`RUNTIME_EXECUTABLE_SEMANTIC_GATE`)

Direct gate operations. `functoriality`

Frozen source locator. Statement lines 2803–2818.

103. Equational realization principle

Stable ID. `PSC-I-T-REALIZATION-EXPORT`

Kind and regime. theorem; algebraic, linear, topological

Statement.

Let $F : \mathbf{C} \to \mathbf{D}$ be an additive functor between preadditive categories. Any equality of morphisms constructed from declared generating morphisms by finite sums, additive inverses, identities, zero maps, and composition is preserved by F. Scalar multiplication by a coefficient ring is also preserved when the categories are enriched over that ring and F is linear for the declared enrichment. If a proof additionally uses a chosen biproduct, kernel, cokernel, quotient, tensor product, or completion and F preserves that construction in its declared profile, then the corresponding proof diagram is transported to $\mathbf{D}$.

Attached source proof. Functoriality gives

$$F(I) = I, \qquad F(g \circ f) = F(g) \circ F(f),$$

and additivity gives

$$F(0) = 0, \qquad F(f + g) = F(f) + F(g), \qquad F(-f) = -F(f).$$

If the declared enrichment is present, linearity additionally gives $F(af) = aF(f)$ for every coefficient a. A structural induction on the construction of a morphism expression therefore carries every permitted equational proof to its image under F. If a step invokes a universal construction, preservation means that the image diagram satisfies the same universal property. The uniquely

induced morphism in the source proof is therefore carried to the uniquely induced morphism in the target proof. Repeating this argument through the finite proof diagram transports the conclusion. □

Recorded dependencies. PSC-I-D-REALIZATION-FUNCTOR

Evidence. AST: TYPED_CORE. Python: EXACT_FINITE_FIXTURE_PASS. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. TRANSPORT_THEOREM (RUNTIME_EXECUTABLE_SEMANTIC_GATE)

Direct gate operations. additive_inverse, composition, finite_sum, functoriality, identity, zero

Frozen source locator. Statement lines 2821–2824; proof lines 2829–2845.

104. Realization export principle

Stable ID. PSC-I-P-REALIZATION-EXPORT

Kind and regime. proposition; algebraic

Statement.

Every canonical Paper I identity whose proof uses only the operations listed in a realization functor's verified preservation profile remains valid after applying that functor. If the proof uses a domain-specific hypothesis not encoded by the functor, that hypothesis must be verified separately before the transported statement is promoted to a theorem.

Attached source proof. The first assertion is PSC-I-T-REALIZATION-EXPORT. The second is a logical boundary: a functor transports established identities and preserved universal constructions, but it does not create an unverified analytic, topological, positivity, finiteness, or integrality hypothesis. □

Recorded dependencies. PSC-I-T-REALIZATION-EXPORT

Evidence. AST: TEXTUAL_SEMANTIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Actions. TRANSPORT_THEOREM (RUNTIME_EXECUTABLE_SEMANTIC_GATE)

Direct gate operations. additive_inverse, composition, finite_sum, functoriality, identity, zero

Frozen source locator. Statement lines 2853–2856; proof lines 2860–2862.

105. Semantic realization criterion

Stable ID. PSC-I-D-SEMANTIC-REALIZATION

Kind and regime. definition; linear

Statement.

A domain theorem is a semantic realization of a canonical seed-calculus theorem only when:

(i) an explicit realization functor or intertwining construction is given;

(ii) the preservation profile covers every construction used in the canonical proof;

(iii) every additional analytic, finiteness, convergence, positivity, or integrality hypothesis is verified in the domain;

(iv) the resulting conclusion is stated in the domain's native objects.

Reuse of the words "seed", "packet", or "transfer" without these data is only syntactic reuse.
Recorded dependencies. None declared.

Evidence. AST: `TYPED_CORE`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `CHECK_REALIZATION_PROFILE` (`RUNTIME_EXECUTABLE_SEMANTIC_GATE`)

Direct gate operations. `functoriality`

Frozen source locator. Statement lines 2864–2874.

A.12 Source section 12: Canonical export interface and claim boundary

106. Reserved example slots

Stable ID. `PSC-I-ALG-EXAMPLE-SLOTS`

Kind and regime. definition; algebraic, linear

Statement.

The later example pass for Paper I is reserved for exactly four canonical realizations:

(i) a typed quiver reachability model;

(ii) an algebraic coordinate-ring presentation;

(iii) a companion and repeated-primary single-operator model;

(iv) path and cycle adjacency models.

The bilateral shift belongs to Paper V, while computational orbit-hull and graph-rank algorithms belong to the verification supplement or Paper VIII. **Recorded dependencies.** None declared.

Evidence. AST: `TEXTUAL_SEMANTIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Frozen source locator. Statement lines 2891–2901.

107. Paper I export object

Stable ID. `PSC-I-EXPORT`

Kind and regime. definition; algebraic, linear, topological

Statement.

The canonical Paper I export is

$$\mathsf{PSC}_I = \left(\begin{array}{c} \text{typed grammars and realized operator algebras,} \\ \text{seed systems, free presentations, and seed rank,} \\ \text{module-top, semisimple, and single-operator formulas,} \\ \text{finite and unconditional packet coordinates,} \\ \text{transfer blocks and support algebras,} \\ \text{reachability and packet-pure source generation,} \\ \text{morphisms, invariant exact sequences, quotients, completions,} \\ \text{realization functors and equational theorem transport} \end{array} \right).$$

Recorded dependencies. None declared.

Evidence. AST: `STRUCTURED_SYMBOLIC`. Python: `NOT_COVERED`. Lean: `NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE`.

Actions. `EXPORT_PSCI_INTERFACE` (`PLANNER_ONLY`)

Frozen source locator. Statement lines 2904–2924.

108. Exact limits of Paper I

Stable ID. PSC-I-B-FINAL-CLAIM-BOUNDARY

Kind and regime. boundary; linear

Statement.

The existence of an exact packet decomposition does not imply that the decomposition is computationally discoverable, numerically well conditioned, statistically identifiable, dynamically invariant, or mathematically useful. The condition number in PSC-I-D-PACKET-CONDITION-NUMBER quantifies only coordinate stability once the packets are supplied. Refinement towers, filtered transfer, nonlinear obstruction objects, automatic packet recovery, continuum limits, arithmetic lifts, and automorphic occupancy require additional theorems in later papers. **Recorded dependencies.** None declared.

Evidence. AST: TEXTUAL_SEMANTIC. Python: NOT_COVERED. Lean: NOT_MAPPED_TO_ATTACHED_SOURCE_TRANCHE.

Boundary metadata. promote_PSCI_export_to_unproved_downstream_domain_theorem

Frozen source locator. Statement lines 2953–2956.

B Typed core signatures

These twenty signatures are reproduced from `kir/core_typed_signatures.json`. Unicode mathematical characters are represented with standard JSON escapes in the listings. Their meanings are unchanged. The schema does not make these strings into independently checked proof terms.

Linear seed system

PSC-I-D-SEED-SYSTEM

```
{
  "params": [
    "A:OperatorAlgebra",
    "M:Module(A)",
    "S:FiniteFamily(M)"
  ],
  "returns": "SeedSystem(A,M,S)"
}
```

Generated hull

PSC-I-D-GENERATED-HULL

```
{
  "params": [
    "A:OperatorAlgebra",
    "M:Module(A)",
    "S:Family(M)"
  ],
  "returns": "Submodule(M)"
}
```

Seed rank

PSC-I-D-SEED-RANK

```
{
  "params": [
    "A:OperatorAlgebra",
    "M:Module(A)"
  ],
  "returns": "Cardinal"
}
```

Finite packet system

PSC-I-D-PACKET-SYSTEM

```
{
  "params": [
    "K:Field",
    "X:VectorSpace(K)",
    "I:FiniteIndexSet",
    "P:I\u2192End(X)"
  ],
  "requires": [
    "idempotent(P)",
    "pairwise_annihilating(P)",
    "sum(P)=id_X"
  ],
```

```
  "returns": "PacketSystem(X,I)"
}
```

Packet analysis and synthesis

PSC-I-D-ANALYSIS-SYNTHESIS

```
{
  "params": [
    "P:PacketSystem(X,I)"
  ],
  "returns": [
    "Analyze_P:X\u2192DirectSum(I,PacketSpace(P,i))",
    "Synthesize_P:DirectSum(I,PacketSpace(P,i))\u2192X"
  ]
}
```

Exact packet-coordinate isomorphism

PSC-I-T-EXACT-PACKET-COORDINATES

```
{
  "params": [
    "P:PacketSystem(X,I)"
  ],
  "returns": "Equivalence(X,DirectSum(I,PacketSpace(P,i)))",
  "certificate": "exact_inverse"
}
```

Transfer block

PSC-I-D-TRANSFER-BLOCK

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "T:End(X)",
    "alpha:I",
    "beta:I"
  ],
  "returns": "LinearMap(PacketSpace(P,alpha),PacketSpace(P,beta))"
}
```

Exact operator reconstruction

PSC-I-T-OPERATOR-RECONSTRUCTION

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "T:End(X)"
  ],
  "returns": "Equality(T,Sum(alpha,beta,TransferBlock(P,T,beta,alpha)))"
}
```

Packet matrix multiplication

PSC-I-T-BLOCK-MULTIPLICATION

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "S:End(X)",
    "T:End(X)",
    "alpha:I",
    "gamma:I"
  ],
  "returns": "Equality(Block(S\u2218T,gamma,alpha),Sum(beta,Block(S,gamma,beta)\u2218Block(T,beta,alpha)))"
}
```

Packet powers and paths

PSC-I-C-BLOCK-POWERS

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "T:End(X)",
    "m:Nat"
  ],
  "returns": "PacketPathExpansion(T,m)"
}
```

Blockwise intertwining equations

PSC-I-T-BLOCK-INTERTWINING

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "Q:PacketSystem(Y,J)",
    "T:End(X)",
    "S:End(Y)",
    "U:LinearMap(X,Y)"
  ],
  "returns": "Iff(U\u2218T=S\u2218U,LocalBlockIntertwining(P,Q,T,S,U))"
}
```

Full block-algebra representation

PSC-I-T-BLOCK-ALGEBRA

```
{
  "params": [
    "P:PacketSystem(X,I)"
  ],
  "returns": "AlgebraEquiv(End(X),DependentBlockAlgebra(P))"
}
```

Support relation and support space

PSC-I-D-SUPPORT-RELATION

```
{
  "params": [
    "I:FiniteIndexSet",
    "R:Relation(I)"
```

```
  ],
  "returns": "SupportSpace(R)"
}
```

Coordinate transport

PSC-I-T-COORDINATE-TRANSPORT

```
{
  "params": [
    "U:LinearEquiv(X,Y)",
    "P:PacketSystem(X,I)",
    "T:End(X)"
  ],
  "returns": "TransportedPacketOperator(U,P,T)"
}
```

Packet exact sequence for an invariant subspace

PSC-I-T-PACKET-EXACT-SEQUENCE

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "K:InvariantSubspace(P)"
  ],
  "returns": "PacketwiseShortExactSequence(K,X,X/K)"
}
```

Packet descent through an invariant quotient

PSC-I-T-QUOTIENT-PACKET-DESCENT

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "K:InvariantSubspace(P)"
  ],
  "returns": "PacketSystem(X/K,I)"
}
```

Naturality of generated hulls and transfer blocks

PSC-I-T-NATURALITY

```
{
  "params": [
    "Phi:PacketCompatibleOperatorMorphism",
    "S:SeedSet"
  ],
  "returns": "NaturalityCertificate(Phi,S)"
}
```

Realization functor and preservation profile

PSC-I-D-REALIZATION-FUNCTOR

```
{
  "params": [
    "F:AdditiveFunctor(C,D)",
    "profile:PreservationProfile"
  ],
  "returns": "RealizationFunctor(C,D,profile)"
}
```

Equational realization principle

PSC-I-T-REALIZATION-EXPORT

```
{
  "params": [
    "Theta:TheoremRecord",
    "F:RealizationFunctor"
  ],
  "requires": [
    "operations_used(Theta)\u2286verified_preservation(F)",
    "target_hypotheses_verified"
  ],
  "returns": "TransportCertificate(Theta,F)"
}
```

Semantic realization criterion

PSC-I-D-SEMANTIC-REALIZATION

```
{
  "params": [
    "Theta:TheoremRecord",
    "F:RealizationFunctor"
  ],
  "returns": "SemanticRealizationCriterion(Theta,F)"
}
```

C Complete action registry

The registry below contains all 37 action names. "Implemented" describes the frozen runtime dispatcher. It does not assert that all hypotheses of every linked source record are automatically validated. Record associations are links for inspection.

AGGREGATE_PACKETS

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-L-PACKET-AGGREGATION

ANALYZE_PACKETS

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_EXACT_FINITE

Records. PSC-I-C-PACKET-UNIVERSAL-PROPERTY; PSC-I-C-PACKETWISE-EQUALITY; PSC-I-D-ANALYSIS-SYNTHESIS; PSC-I-T-EXACT-PACKET-COORDINATES

ASSEMBLE_BLOCK_ALGEBRA

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-DIMENSION-COUNT; PSC-I-T-BLOCK-ALGEBRA

BLOCKIFY_OPERATOR

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_EXACT_FINITE

Records. PSC-I-D-TRANSFER-BLOCK; PSC-I-T-OPERATOR-RECONSTRUCTION

BUILD_SEED_HULL

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-D-GENERATED-HULL; PSC-I-D-SEED-EVALUATION

BUILD_SUPPORT_ALGEBRA

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-SUPPORT-PATTERN-STAR; PSC-I-T-PREORDER-ALGEBRA-DECOMPOSITION; PSC-I-T-SUPPORT-PATTERN-ALGEBRA

BUILD_TRANSFER_GRAPH

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-D-TRANSFER-GRAPH

CHANGE_PACKET_COORDINATES

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY

Records. PSC-I-T-COORDINATE-TRANSPORT

CHECK_REALIZATION_PROFILE

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_SEMANTIC_GATE

Records. PSC-I-D-REALIZATION-FUNCTOR; PSC-I-D-SEMANTIC-REALIZATION

COMBINE_SEED_SYSTEMS

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-P-SHARED-DIRECT-SUM; PSC-I-P-TENSOR-BOUND; PSC-I-T-EXACT-SEQUENCE-BOUNDS; PSC-I-T-INDEPENDENT-DIRECT-SUM

COMPLETE_PACKET_SPLIT

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-COMPLETION-PACKET-SPLIT; PSC-I-T-COUNTABLE-UNCONDITIONAL-PACKETS

COMPLETE_TOPOLOGICAL_REALIZATION

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-TOPOLOGICAL-PACKET-EXACT-SEQUENCE; PSC-I-T-DENSE-REALIZATION; PSC-I-T-QUOTIENT-TOPOLOGY-COMPLETION

COMPOSE_TRANSFER_BLOCKS

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_EXACT_FINITE

Records. PSC-I-C-TENSOR-TRANSFER-BLOCKS; PSC-I-T-BLOCK-MULTIPLICATION

COMPUTE_PACKET_PURE_RANK

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-C-SOURCE-COMPONENT-COUNT; PSC-I-D-DIRECTED-SEED-RANK; PSC-I-P-PACKET-PURE-RANK-BOUNDS; PSC-I-T-SOURCE-COMPONENT-SEED-RANK

COMPUTE_REACHABILITY

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-P-REACHABILITY-OBSTRUCTION; PSC-I-T-ALGEBRA-REACHABILITY-SUPPORT

COMPUTE_SEED_RANK

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-C-ALGEBRAICALLY-CLOSED-SINGLE-OPERATOR; PSC-I-C-INVARIANT-FACTOR-COUNT; PSC-I-D-DIRECTED-SEED-RANK; PSC-I-D-SEED-RANK; PSC-I-P-CYCLIC-CHARACTERIZATION; PSC-I-T-MODULE-TOP; PSC-I-T-SEMISIMPLE-SEED-RANK; PSC-I-T-SINGLE-OPERATOR; PSC-I-T-SOURCE-COMPONENT-SEED-RANK

DECOMPOSE_PRIMARY

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-ALGEBRAICALLY-CLOSED-SINGLE-OPERATOR; PSC-I-C-CYCLIC-MATRIX; PSC-I-C-INVARIANT-FACTOR-COUNT; PSC-I-T-SINGLE-OPERATOR

DESCEND_PACKET_QUOTIENT

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY

Records. PSC-I-C-QUOTIENT-PACKET-DESCENT; PSC-I-T-QUOTIENT-PACKET-DESCENT

DUALIZE_PACKETS

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-P-DUAL-PACKET-SYSTEM

EVALUATE_PACKET_CONDITIONING

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-D-PACKET-CONDITION-NUMBER; PSC-I-P-CONDITIONED-COORDINATES

EXPAND_OPERATOR_PATHS

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY

Records. PSC-I-C-BLOCK-POWERS

EXPORT_PSCI_INTERFACE

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-EXPORT

EXTRACT_PACKET_COMMUTANT

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-P-BLOCK-DIAGONAL-COMMUTANT

FACTOR_ACTION

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-P-ACTION-FACTORIZATION

LOWER_INTERTWINER

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_EXACT_FINITE

Records. PSC-I-T-BLOCK-INTERTWINING

PRESENT_AS_FREE_QUOTIENT

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-C-MINIMAL-FREE-COVER; PSC-I-C-ONE-SEED-PRESENTATION; PSC-I-T-FREE-PRESENTATION

PROPAGATE_PACKET_SUPPORT

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY

Records. PSC-I-C-PACKET-SUPPORT-PROPAGATION

PUSH_GENERATORS_THROUGH_QUOTIENT

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-P-SCALAR-QUOTIENT-WITNESS; PSC-I-T-QUOTIENT-MONOTONICITY

REALIZE_GRAMMAR

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-D-COLLAPSED-REALIZATION; PSC-I-T-TYPED-MODULE-EQUIVALENCE

REDUCE_TO_SEMISIMPLE_TOP

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-LOCAL-ALGEBRA-FORMULA; PSC-I-T-MODULE-TOP; PSC-I-T-SEMISIMPLE-SEED-RANK

RESOLVE_PACKET_EXACT_SEQUENCE

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-C-TOPOLOGICAL-PACKET-EXACT-SEQUENCE; PSC-I-T-PACKET-EXACT-SEQUENCE

TENSOR_PACKETS

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY

Records. PSC-I-T-TENSOR-PACKET-SYSTEM

TRANSPORT_MORPHISM

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-D-PACKET-MORPHISM; PSC-I-D-SEED-MORPHISM; PSC-I-P-SEED-SYSTEM-CATEGORY; PSC-I-T-NATURALITY

TRANSPORT_SEED_RANK

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-ISOMORPHISM-INVARIANCE; PSC-I-P-SCALAR-EXTENSION-MONOTONICITY

TRANSPORT_THEOREM

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_SEMANTIC_GATE

Records. PSC-I-P-REALIZATION-EXPORT; PSC-I-T-REALIZATION-EXPORT

VALIDATE_PACKET_SYSTEM

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_EXACT_FINITE

Records. PSC-I-D-PACKET-SYSTEM

VERIFY_PROPAGATION_HYPOTHESES

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-D-EDGE-SURJECTIVE; PSC-I-D-PACKET-PROPAGATING

D Operation and hypothesis vocabulary

The frozen ontology separates structural operations from hypothesis classes. These names form a controlled vocabulary for profiles and source-derived annotations. They are not a complete formal semantics for every mathematical construction named.

Name	Class
`identity`	structural
`zero`	structural
`finite_sum`	structural
`additive_inverse`	structural
`composition`	structural
`scalar_action`	structural
`direct_sum`	structural
`finite_product`	structural
`biproduct`	structural
`kernel`	structural
`cokernel`	structural
`image`	structural
`quotient`	structural
`tensor_product`	structural
`dual`	structural
`packet_idempotents`	structural
`transfer_blocks`	structural
`support_relation`	structural
`path_composition`	structural
`polynomial_action`	structural
`jacobson_radical`	structural
`semisimple_decomposition`	structural
`primary_decomposition`	structural
`universal_property`	structural
`functoriality`	structural
`exact_sequence`	structural
`closure`	structural
`completion`	structural
`topology`	structural
`norm`	structural
`adjoint`	structural
`dimension`	structural
`finite_indexing`	hypothesis
`surjectivity`	hypothesis
`injectivity`	hypothesis
`isomorphism`	hypothesis
`continuity`	hypothesis
`boundedness`	hypothesis
`closedness`	hypothesis
`density`	hypothesis
`orthogonality`	hypothesis
`self_adjointness`	hypothesis
`positivity`	hypothesis
`integrality`	hypothesis

Name	Class
unconditional_convergence	hypothesis
hausdorff	hypothesis
complete_field	hypothesis
invariant_subspace	hypothesis
commutation	hypothesis
transitivity	hypothesis
edge_surjectivity	hypothesis
packet_propagation	hypothesis
nonzero_packet_spaces	hypothesis

E Historical Lean record map

This is the eight-entry mapping attached to the frozen machine release. The evidence descriptions and source-match categories are reproduced from that map. A fresh build performed elsewhere must be documented separately.

Finite packet system

PSC-I-D-PACKET-SYSTEM **Source match.** `structural_exact`

Stored certificate status. `compiled_in_verified_tranche`

Declarations. `Komorebi.Foundation.LinearProjectorFamily`

ProjectorCore.lean is part of the v0.2.0 kernel-accepted project; structure axioms encode idempotence, pairwise annihilation, and completeness.

Packet analysis and synthesis

PSC-I-D-ANALYSIS-SYNTHESIS **Source match.** `stronger_typed_realization`

Stored certificate status. `kernel_accepted`

Declarations. `Komorebi.Foundation.LinearProjectorFamily.PacketSpace`; `Komorebi.Foundation.LinearProjectorFamily.rangeAnalysis`; `Komorebi.Foundation.LinearProjectorFamily.rangeSynthesis`

ExactCoordinates.lean defines the packet space as the product of projector ranges, so both transforms carry range certificates.

Exact packet-coordinate isomorphism

PSC-I-T-EXACT-PACKET-COORDINATES **Source match.** `stronger_typed_realization`

Stored certificate status. `kernel_accepted`

Declarations. `Komorebi.Foundation.LinearProjectorFamily.exactPacketCoordinates`

Explicitly listed as Kernel accepted in VERIFICATION_REPORT.md; proves both inverse laws for range-restricted packet coordinates.

Packetwise equality

PSC-I-C-PACKETWISE-EQUALITY **Source match.** `one_direction_plus_trivial_reverse`

Stored certificate status. `compiled_in_verified_tranche`

Declarations. `Komorebi.Foundation.LinearProjectorFamily.eq_of_projectors_eq`

The nontrivial direction is proved in ProjectorCore.lean; the reverse direction follows by congruence from equality.

Transfer block

PSC-I-D-TRANSFER-BLOCK **Source match.** `generalized_domain_codomain`

Stored certificate status. `compiled_in_verified_tranche`

Declarations. `Komorebi.Foundation.transferBlock`

Definition allows distinct source and target packet systems; the PSC-I endomorphism case is a specialization.

Exact operator reconstruction

PSC-I-T-OPERATOR-RECONSTRUCTION **Source match.** `generalized_domain_codomain`

Stored certificate status. `kernel_accepted`

Declarations. `Komorebi.Foundation.transferBlock_reconstruction`

Explicitly listed as Kernel accepted; reconstructs any linear map from all source-target blocks using completeness of both packet families.

Packet matrix multiplication

PSC-I-T-BLOCK-MULTIPLICATION **Source match.** `generalized_composition_form`

Stored certificate status. `kernel_accepted`

Declarations. `Komorebi.Foundation.transferBlock_comp`

Explicitly listed as Kernel accepted; inserts the complete middle packet family and yields the local composition sum.

Support-pattern algebra criterion

PSC-I-T-SUPPORT-PATTERN-ALGEBRA **Source match.** `partial_sufficient_direction`

Stored certificate status. `kernel_accepted_related`

Declarations. `Komorebi.Algebra.MatrixSupported.mul`

Verified tranche proves multiplicative closure for transitive support relations; it is related evidence, not a source-equivalent certificate for every direction of the PSC-I criterion.

F Current verification receipt

The following is the successful current run of the documented frozen release gate. The detailed exact-fixture output, command lines, interpreter versions, earlier missing-dependency explanation, and frozen-byte audit are retained in the build results.

```
ENTRYPOINT_VERSION=PASS
RECORD_COUNT=PASS
DEPENDENCY_ACYCLIC=PASS
DEPENDENCIES_RESOLVE=PASS
AST_COVERAGE=PASS
ACTION_REGISTRY=PASS
SOURCE_HASH=PASS
API_STATUS=PASS
API_RECORD=PASS
API_DEPS=PASS
API_AST=PASS
EXECUTE_BLOCKIFY=PASS
EXECUTE_PACKET_VALIDATE=PASS
TRANSPORT_GATE_STRUCTURED=PASS
EXACT_FIXTURE_REPLAY=PASS
PSC_I_MACHINE_V1_RELEASE_GATE=PASS
```

Interpreter provenance

```
3.12.14 (main, Aug 25 2026, 14:00:49) [Clang 22.1.3 ]
SymPy=1.14.0
```

Source and package preservation

The companion's integrity audit compared each of the 65 original archived files with the copied payload. Every comparison passed. Generated Python bytecode caches, if any, are not part of the frozen release and are excluded from the delivered package.

Proofreading scope. The editorial prose, worked displays, generated links, tables, and record rendering were reviewed for local consistency. Source statements and proofs were reproduced from the corrected v1.0.1 records. The two audited nonzero-packet additions are the only mathematical-token changes relative to v1.0.0. This editorial review does not constitute a new mathematical proof audit.